Week 3 — C Memory, Compilation, Linking, and Program Startup

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

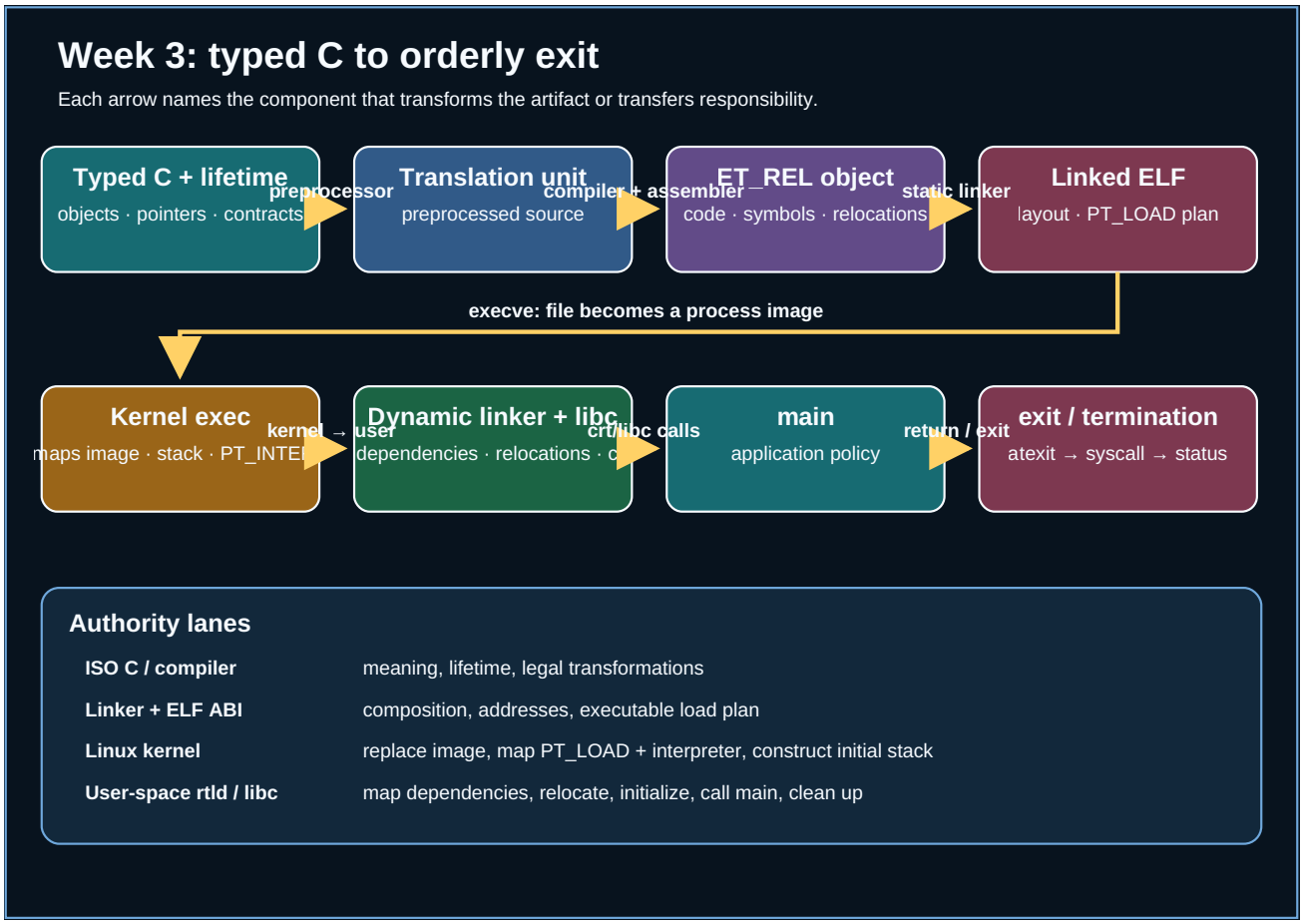
> A C expression begins with typed objects and lifetime rules. A running process begins with mapped machine code, startup state, and a kernel-to-user handoff. This week connects those views without skipping the compiler, object file, linker, ELF loader, or C runtime between them.

“Week” names a mastery module, not a calendar deadline. Spend longer when a sanitizer report, relocation, or startup trace exposes a weak link. Familiar material may take less time, but completion still requires evidence across Explain, Draw, Observe, and Build.

Why these seven days belong together

The first four days define what a C program is allowed to mean. The final three show how that meaning becomes a Linux process:

```
typed C objects, pointers, aggregates, and callbacks
        ↓ lifetime, ownership, qualifiers, atomics, and defined behavior
translation unit
        ↓ compiler and assembler
relocatable ELF: sections, symbols, machine code, relocations
        ↓ static linker
executable ELF: resolved layout plus dynamic-link requirements
        ↓ execve and kernel ELF loading
mapped executable, interpreter, initial stack, and auxiliary vector
        ↓ user-space dynamic linker and libc startup
relocations → constructors → main → exit/atexit → kernel termination
```



Responsibility changes at every boundary: language, toolchain, kernel, runtime, application, then kernel again.

The connections matter more than the list. A pointer names an object whose lifetime and type constrain valid accesses. The optimizer relies on those constraints while producing instructions. Separate translation leaves symbols and address calculations unfinished. The linker composes objects. ELF program headers then describe a process image; Linux and the user-space dynamic linker perform different parts of startup before libc calls `main`.

Daily roadmap and exact files

Day 1 — Pointers, Arrays, and Strings

Define an address precisely, derive typed pointer arithmetic, distinguish arrays from pointers and array decay, reason about multidimensional layout, and treat C strings as a length/lifetime safety problem rather than as a built-in type.

Lesson:

```
03-c-toolchain-and-startup/sections/day-001-pointers-arrays-and-strings.md
```

Challenges:

```
03-c-toolchain-and-startup/challenges/day-001-pointer-and-array-lab.c
03-c-toolchain-and-startup/challenges/day-001-string-functions-lab.c
```

Evidence: predicted and observed addresses/strides, array-versus-pointer size results, ASLR comparison, completed string primitives, and annotated AddressSanitizer reports.

Day 2 — Structures, Unions, and Function Pointers

Predict structure padding and alignment, use tagged unions safely, understand callbacks and context pointers, build operations tables and opaque handles, and connect intrusive containers to kernel-style interfaces.

Lesson:

```
03-c-toolchain-and-startup/sections/day-002-structures-unions-and-function-pointers.md
```

Challenge:

```
03-c-toolchain-and-startup/challenges/day-002-layout-callback-lab.c
```

Evidence: hand-predicted and observed member offsets, one disassembled member access, one indirect call, a callback registry, and a miniature operations table.

Day 3 — Dynamic Memory and Lifetime

Separate scope, storage duration, object lifetime, reachability, and ownership. Use the allocation family with overflow and failure handling, design cleanup paths, inspect memory errors, and build an arena whose bulk lifetime is explicit.

Lesson:

```
03-c-toolchain-and-startup/sections/day-003-dynamic-memory-and-lifetime.md
```

Challenges:

```
03-c-toolchain-and-startup/challenges/day-003-arena-lab.c
03-c-toolchain-and-startup/challenges/day-003-memory-errors.c
```

Evidence: arena offsets and capacity behavior, one sanitizer report, one leak report where available, and an ownership table naming every allocated object's owner, borrowers, lifetime end, and cleanup path.

Day 4 — Qualifiers, Atomics, and Undefined Behavior

Use `const`, `volatile`, and `restrict` under their actual contracts; distinguish device observation from inter-thread synchronization; reason about sequence and undefined behavior; and introduce C atomics as operations with memory-order guarantees rather than “volatile but stronger.”

Lesson:

```
03-c-toolchain-and-startup/sections/day-004-qualifiers-atomics-and-undefined-behavior.md
```

Challenges:

```
03-c-toolchain-and-startup/challenges/day-004-qualifiers-ub-lab.c
03-c-toolchain-and-startup/challenges/day-004-atomics-lab.c
```

Evidence: `-O0` / `-O2` assembly comparisons, sanitizer diagnostics for selected invalid modes, and repeated volatile-versus-atomic observations with the compiler, CPU, and run count recorded.

Day 5 — From Source to Object File

Stop GCC or Clang after preprocessing, compilation, and assembly. Distinguish translation units, AST/IR concepts, assembly, and ELF relocatable objects; then inspect sections, symbols, machine instructions, and relocation records before final linking.

Lesson:

```
03-c-toolchain-and-startup/sections/day-005-source-to-object-file.md
```

Challenges:

```
03-c-toolchain-and-startup/challenges/day-005-object-demo.c
03-c-toolchain-and-startup/challenges/day-005-object-lab.py
```

Evidence: one macro expansion, one optimization-driven assembly change, one defined and one undefined symbol, one instruction paired with its relocation, and a separately compiled provider that completes the program.

Day 6 — Linking and Libraries

Resolve strong, weak, local, and undefined symbols; apply relocations after layout; compare archive selection with shared-object dependencies; and inspect PIC, GOT/PLT behavior, SONAME, RUNPATH, and dynamic binding without collapsing build-time and runtime linking.

Lesson:

```
03-c-toolchain-and-startup/sections/day-006-linking-and-libraries.md
```

Challenge:

```
03-c-toolchain-and-startup/challenges/day-006-linking-lab.sh
```

Evidence: archive and shared builds, `DT_NEEDED`/SONAME/RUNPATH output, one dynamic relocation, one PLT/GOT observation, and diagnosed undefined and duplicate symbol failures.

Day 7 — ELF Loading and `main`

Distinguish ELF sections from loadable segments; cross the successful `execve` boundary; trace kernel mappings and the `PT_INTERP` handoff; then follow the user-space dynamic linker, relocations, crt/libc startup, `argc` / `argv` / `envp` /auxv, constructors, `main`, `exit`, and `atexit`.

Lesson:

```
03-c-toolchain-and-startup/sections/day-007-elf-loading-and-main.md
```

Challenges:

```
03-c-toolchain-and-startup/challenges/day-007-startup-probe.c
03-c-toolchain-and-startup/challenges/day-007-elf-startup-lab.sh
```

Evidence: ELF entry and program headers, section-to-segment mapping, `PT_INTERP`, `_start` disassembly, labelled `/proc/self/maps`, selected auxv values, constructor/`main`/`atexit` order, and a bounded trusted `LD_DEBUG` trace.

Adjustable workload guide

The nominal budget is approximately 21 focused hours:

```
Day 1   3 h   address model, arrays/strings, sanitizers
Day 2   3 h   layout, unions, callbacks, operations tables
Day 3   3 h   ownership, allocation, arena, diagnostics
Day 4   3 h   qualifiers, optimizer freedom, atomics, UB
Day 5   3 h   translation stages and relocatable ELF
Day 6   3 h   symbol resolution, relocation, libraries, GOT/PLT
Day 7   3 h   execve, mappings, dynamic loader, libc startup and exit
```

This is a planning estimate, not a completion rule:

- split sanitizer, GDB, or dynamic-loader work across sessions if attention drops;
- repeat an experiment when observation contradicts prediction;
- shorten exposition already understood, but still preserve evidence;
- record unavailable static libraries, debugger behavior, or sanitizer runtimes as environment facts rather than silently skipping them;
- never mark a day mastered only because its Markdown was read.

Integrated weekly evidence

Explain

Without notes, tell one connected story:

1. a pointer is a typed way to name storage, and arrays/strings add layout and termination conventions;
2. structures, unions, callbacks, and operations tables organize state and runtime-selected behavior;
3. ownership and lifetime decide when an address may validly be used;
4. qualifiers, atomics, and the C abstract machine decide what observations and transformations are permitted;
5. preprocessing, compilation, and assembly create an ELF object containing code/data, symbols, and unfinished relocations;
6. the linker resolves names, lays out output, applies static relocations, and records remaining dynamic requirements;
7. Linux uses program headers to replace a process image and enter user code;
8. the user-space dynamic linker and libc complete startup before `main`;
9. normal return from `main` passes through libc cleanup and a kernel termination boundary.

At each step name whether the governing rule comes from ISO C, the compiler, the ABI, ELF, the static linker, the Linux kernel, the dynamic linker, libc, or application policy.

Draw

Produce one source-to-execution diagram containing:

- C objects, pointers, owners, and lifetime boundaries;
- one structure layout and one callback through an operations table;
- source → translation unit → assembly → relocatable object;
- object sections, symbol table, and a relocation attached to an instruction;
- linker selection/layout and archive-versus-shared paths;
- executable sections grouped into `PT_LOAD` segments;
- process mappings for executable, interpreter, libc, heap, and stack;
- the initial `argc` / `argv` / `envp` /auxv stack;
- kernel and user-space responsibility lanes through `main` and exit.

Use arrows labelled with the responsible component. “System” or “loader” without a kernel/user-space distinction is not enough.

Observe

Keep a compact, reproducible evidence bundle:

- pointer/array outputs and sanitizer reports;
- structure offsets and one indirect callback instruction;
- arena behavior and one lifetime-error diagnostic;
- qualifier/atomic assembly or repeated-run results;
- preprocessed source, object symbols, sections, and relocations;
- archive/shared dependency and binding evidence;
- ELF program headers and section-to-segment mapping;
- `/proc/self/maps`, auxv, constructor/`atexit` output, and `LD_DEBUG`;
- compiler, linker/binutils, libc, kernel, and architecture versions.

Sanitizer reports and loader traces can contain addresses that vary under ASLR. Preserve enough surrounding information to explain the relationship, not to memorize one address.

Build

Complete or extend the challenge programs so the final code demonstrates:

- safe pointer-plus-length handling and string bounds;
- a tagged union and callback/operations-table interface;
- explicit ownership and an arena lifetime;
- an atomic communication path with stated memory ordering;
- a multi-translation-unit program stopped and inspected at object stage;
- static archive and shared-object forms of one interface;
- a startup probe showing constructor, `main`, auxv/maps, `atexit`, and the expected exit status.

Every extension should retain the loop:

```
predict → build/run → preserve evidence → explain the responsible contract
```

Integrated source-to-execution mastery checkpoint

Start with a two-file C program in which `main`:

1. allocates an array of structures;
2. fills it through a callback selected from an operations table;
3. calls a function provided by a shared library;
4. registers an `atexit` handler;
5. returns a nonzero status.

Then explain the complete route:

1. What are the array object's size, alignment, owner, aliases, and lifetime?
2. Which pointer operations are defined, and where must a count accompany a pointer?
3. How is the callback type checked, and what machine-level event performs the indirect call?
4. Which accesses require atomics if another thread participates, and why is `volatile` insufficient?
5. What does each source file become after preprocessing, compilation, and assembly?
6. Which symbols are defined or undefined in each `.o`, and which relocations remain?
7. How does the static linker resolve local objects while preserving the shared-library requirement?
8. Which ELF sections contribute to which loadable segments?
9. After successful `execve`, what does Linux map and place on the initial stack?
10. Why does execution commonly enter the interpreter before executable `_start`?
11. Who maps the shared library and applies its runtime relocations?
12. How do `_start`, libc startup, constructors, and the ABI lead to `main`?
13. What happens to the allocation and registered handler when `main` returns, and where does kernel responsibility resume?

Mastery is demonstrated when every arrow is explicit:

```
C meaning and lifetime
  → compiler representation and optimization
  → object code, symbols, and relocations
  → linker composition
  → ELF load plan
  → kernel exec setup
  → user-space dynamic linking and libc startup
  → main
  → orderly exit and kernel termination
```

If an answer says “the compiler,” “the linker,” or “the loader” without identifying which artifact changed and which component had authority, return to that boundary and inspect the evidence again.

End-of-module reflection

Record in your own words:

- strongest understanding;
- remaining confusion;
- one prediction contradicted by actual output;
- connection to Linux, kernel, driver, or virtualization work;
- one boundary you can now explain more precisely;
- next revision action;
- whether to advance, repeat one lab, or extend this module.

Do not auto-fill confidence, completion, hours, or mastery. Those remain the learner's evidence-based assessment.

Day 1 — Addresses You Have Already Seen

Curriculum alignment: Stage 5 — C from the Machine's Perspective · Pointers, Arrays, and Strings

Target time: approximately 3 hours

- Retrieval and the address model: about 30 minutes
- Pointer arithmetic and array decay: about 45 minutes
- Lab A, prediction and observation: about 30 minutes
- Strings, and why they are shaped this way: about 35 minutes
- Lab B, implementing the primitives: about 30 minutes
- Mastery work: about 20 minutes

> Last week you watched a C function turn into `mov`, `lea`, `push`, and `call`. You saw a stack frame built and torn down. You saw `mov 0x8(%rbp),%eax` reach into memory and bring a value back.
>
> So here is today's only real question:
>
> `text what new thing does a "pointer" add to a machine that could already do that?`
>
> The honest answer is: almost nothing. The machine was already using addresses. C is just letting you hold one.

---

1. You already know what a pointer is; you have not been allowed to say so

Recall the shape of a memory access you disassembled last week:

```
mov    0x8(%rbp), %eax
```

Three things are present in that one instruction.

1. `%rbp` holds a **number**.
2. That number plus 8 identifies a **location** in memory.
3. The instruction reads four bytes **from** that location.

The number in `%rbp` is an address. It is an ordinary 64-bit integer as far as the register file is concerned — the same register could hold a loop counter next instruction. What makes it an address is not its bits but the fact that this instruction used it as one.

You also know the memory model behind it. Physical DRAM is organised into rows and columns; the memory controller receives a number and returns the contents of the cell that number selects. Between the CPU and DRAM sit caches, and last week you measured what

happens when your access pattern ignores them. Above all of that sits the virtual-address translation you have not studied yet, which is why every process believes it owns the same address space.

Here is the point. Every one of those layers takes **a number that identifies a location**. C's contribution is a type system that lets a program compute and store those numbers safely, and a set of rules for what you may do with them.

So define the word carefully:

> A **pointer** is a value whose type says "this is the address of an object of type `T`." On this machine its representation is a 64-bit address, but its *type* is what gives it meaning.

The distinction between representation and type will decide the rest of today. Two pointers with identical bit patterns can be different C objects. One pointer can be perfectly valid at 10:00 and undefined at 10:01 without a single bit changing.

Why the language needed the concept at all

Three problems forced pointers into every systems language.

Problem one: a function cannot change what it cannot reach. C passes arguments by value. `void twice(int x) { x *= 2; }` modifies a copy that lives in the callee's frame. If the caller wants its own variable changed, it must hand over a way to find that variable — its address.

Problem two: copying is expensive and sometimes wrong. Passing a 4-kilobyte structure by value means copying 4 kilobytes onto the stack on every call. Passing its address means copying eight bytes. And if two parts of a program must agree about *the same* object rather than about equal values, only an address can express that.

Problem three: data structures need to refer to each other. A linked list, a tree, a hash table with chaining, and a kernel's list of runnable tasks all need "this record is connected to that record." At machine level the only way to say that is to store the other record's address.

None of this is C-specific. It is what the hardware forces on anybody who wants to build data structures in a flat, numbered memory.

What a 64-bit address actually is on this machine

Print `sizeof(void *)` here and you get 8. But not all 64 bits are usable. x86-64 in its common configuration uses 48-bit virtual addresses, sign-extended into the top bits, which is why user-space addresses on this VM look like:

```
0x00007ffe07cf6dfc     a stack address
0x0000000000403074     a .bss address in a non-PIE executable
0x00007f627db3c000     a shared-library mapping
```

The kernel occupies the other half, with the top bits all ones. An address with a mismatched top half is *non-canonical* and faults on use. You do not need that detail today, but it explains why pointer values on Linux cluster into recognisable neighbourhoods rather than spreading across the full 64-bit range.

Architecture lens — x86-64, AArch64, and RISC-V

Common 64-bit Linux ABIs for all three targets use eight-byte object pointers, but ISO C does not require that width or require every pointer value to be a usable integer address. The implemented virtual-address range is also smaller than 64 bits and varies by CPU and configuration: x86-64 canonical address rules are not AArch64 or RISC-V rules.

Keep pointer arithmetic, bounds, lifetime, and alignment reasoning in the C abstract machine. The compiler then maps valid accesses to the target ISA and ABI; hardware tolerance of an unaligned access, or a numerically plausible address, does not make an invalid C access defined. See the architecture comparison for the target-level distinctions.

---

2. Why `p + 1` does not add one

Write a function that advances a pointer and look at what the compiler emits. Three types, three different constants:

```
int        *bump_int(int *p)         { return p + 1; }
char       *bump_char(char *p)       { return p + 1; }
struct big *bump_big(struct big *p)  { return p + 1; }   /* 12 bytes */
```

Compiled with `gcc -O2 -S` on this machine:

```
bump_int:     leaq   4(%rdi), %rax
bump_char:    leaq   1(%rdi), %rax
bump_big:     leaq  12(%rdi), %rax
```

The `+ 1` in the source became `+4`, `+1`, and `+12`. The compiler multiplied by `sizeof(*p)` before emitting the instruction.

This is not a convenience feature bolted on afterwards. It falls out of what pointer arithmetic is *for*. Pointers move through arrays, arrays store elements end to end, and the interesting question is almost always "give me the next element," not "give me the next byte." The language made the common case the default.

The inverse operation is symmetric. Subtracting two pointers of the same type yields how many **elements** apart they are:

```
long diff(int *a, int *b) { return a - b; }
```

```
diff:   subq   %rsi, %rdi
        movq   %rdi, %rax
        sarq   $2, %rax
```

Subtract the byte addresses, then arithmetic-shift right by 2 — divide by `sizeof(int)`. The type of that result is `ptrdiff_t`, which is signed, because `b` may be after `a`.

Indexing is arithmetic in disguise

The C standard defines `E1[E2]` as `(*((E1)+(E2)))`. That definition, not a special array-access instruction, is the whole of indexing. It has one famous consequence:

```
arr[2]  ==  *(arr + 2)  ==  *(2 + arr)  ==  2[arr]
```

All four are the same expression after the definition is applied. Lab A prints all four and gets 30 from each. Never write the fourth form, but do let it convince you that the subscript operator is sugar.

And when the index is a constant, the "arithmetic" happens at compile time:

```
int index3(int *p) { return p[3]; }
```

```
index3: movl   12(%rdi), %eax
```

There is no multiply and no add at run time. The displacement field of the addressing mode you met last week absorbed the whole computation. This is the moment where last week's `disp(base,index,scale)` form stops being trivia: it exists precisely because array indexing is the most common thing programs do to memory.

The rules that keep this honest

Pointer arithmetic is only defined inside an object, plus one position past its end:

```
int a[5];

a, a+1, a+2, a+3, a+4    valid, and dereferenceable
a+5                      valid to FORM and compare, not to dereference
a+6                      undefined behaviour, even if you never deref it
a-1                      undefined behaviour
```

The one-past-the-end allowance exists so that loop conditions like `for (p = a; p != a + 5; ++p)` are legal. It is the smallest concession that makes iteration expressible.

Why is `a + 6` undefined rather than merely useless? Because the standard declines to assume memory is flat and unbounded. Segmented architectures existed; computing an address outside an object could overflow a segment offset. More relevantly today, the rule lets the optimiser reason: if the compiler knows `p` points into `a`, it knows `p - a` fits in a small range, and that enables loop transformations. You saw the same bargain last week with signed overflow, and you will see it again on Day 4.

---

3. An array is not a pointer, and the difference is where the length lives

This is the most persistently misunderstood corner of C, so let us build it rather than assert it.

An array is an object. `int a[5]` is twenty contiguous bytes with one address and a size the compiler knows. `sizeof a` is 20.

A pointer is also an object — eight bytes holding an address. `sizeof p` is 8.

They are different types with different sizes. What confuses everyone is a conversion rule:

> In most expression contexts, an expression of array type is converted to a pointer to its first element. This is called **decay**.

"Most" has exceptions, and the exceptions are precisely the places where the array's identity still matters: the operand of `sizeof`, the operand of `&`, and a string literal used to initialise a `char` array.

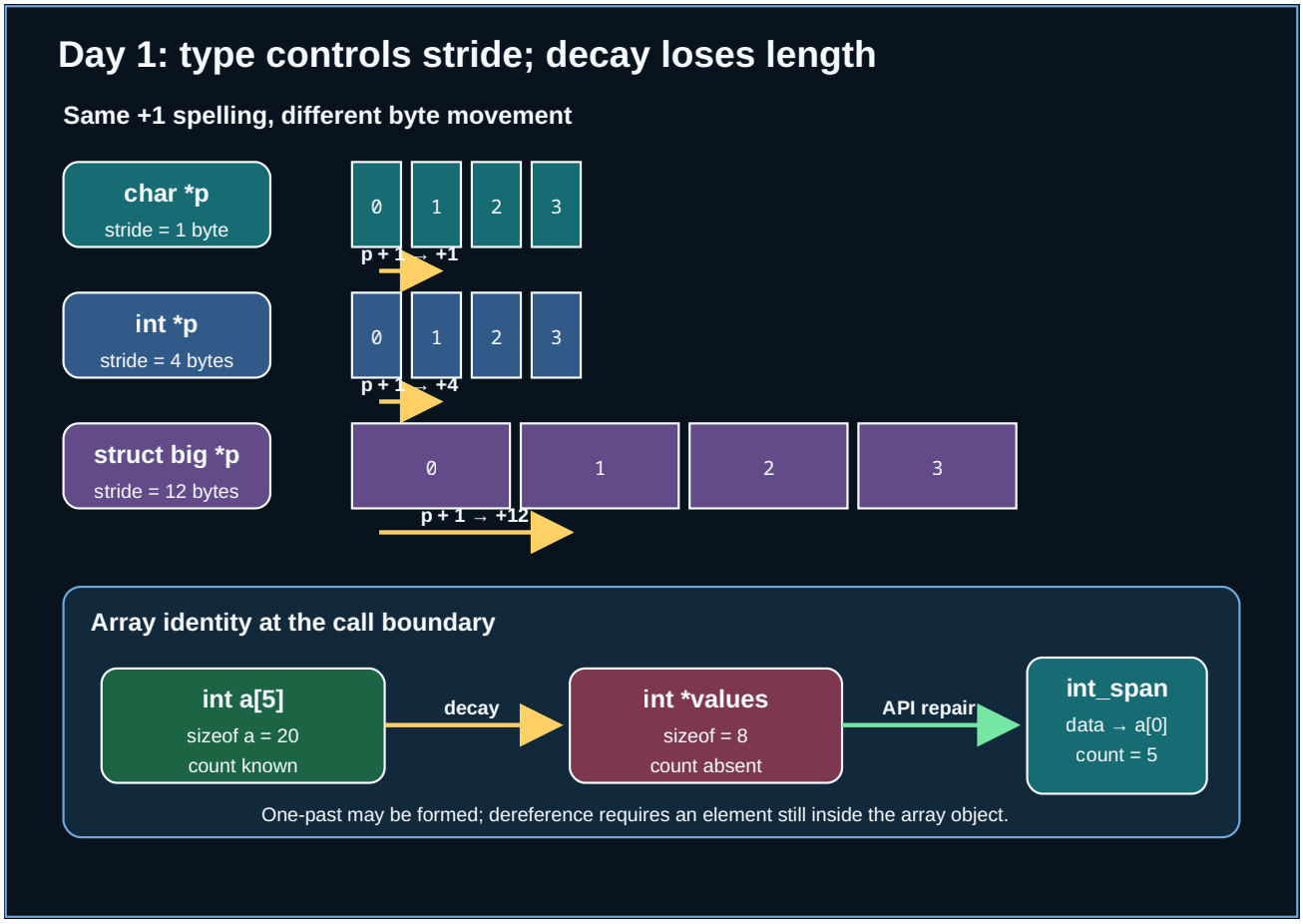
Lab A prints all three spellings of the same location:

```
arr        = 0x7ffca1ec4cf0   sizeof = 20
&arr[0]    = 0x7ffca1ec4cf0   sizeof = 4
&arr       = 0x7ffca1ec4cf0   sizeof = 8
arr + 1    = 0x7ffca1ec4cf4   (one int later)
&arr + 1   = 0x7ffca1ec4d04   (one whole array later)
```

Three identical addresses, three different types:

```
arr      decays to  int *          → +1 moves 4 bytes
&arr[0]  is         int *          → same thing, written explicitly
&arr     is         int (*)[5]     → +1 moves 20 bytes
```

`0x...cf0 + 20 = 0x...d04`. The pointer type, not the address, decided the stride.



The pointer type preserves stride; array decay preserves an address but loses the element count.

Decay is a design decision with a cost

Why would a language throw away the length like that?

Partly history: C inherited from BCPL and B, languages in which a vector was essentially a pointer to a block. Partly efficiency, and this part still holds: if arrays were passed by value, calling `process(big_array)` would copy the whole thing. Decay made the cheap behaviour the default behaviour, in an era when a PDP-11 had 64 KiB of address space per process.

The cost is permanent and you pay it every day:

```
  inside caller: sizeof(arr) = 20, count = 5
  inside callee: sizeof(values) = 8 (a pointer)
  inside callee: caller had to pass count = 5 separately
```

A function receiving `int *values` cannot recover the length. Not "finds it difficult" — the information is not present. Every bounds bug in C descends from this one fact. Every API you design must answer it explicitly, by passing a count, by using a sentinel, or by wrapping the pointer and length in a structure.

That last option is what modern C leans toward:

```
struct int_span {
    int   *data;
    size_t count;
};
```

You will see the same shape in Rust's slices, Go's slices, C++'s `std::span`, and in the kernel's many `(buf, len)` argument pairs. The idea everyone converged on is: *keep the length adjacent to the pointer*. C simply never made it a language feature.

Two-dimensional arrays are one-dimensional with extra typing

```
int grid[3][4];
```

This is "an array of 3 elements, each of which is an array of 4 `int`." Lab A confirms the layout:

```
sizeof(grid)       = 48
sizeof(grid[0])    = 16
sizeof(grid[0][0]) = 4
&grid[0][0] = 0x7ffca1ec4cd0
&grid[1][0] = 0x7ffca1ec4ce0  (row stride = 16 bytes)
flat walk: 0 1 2 3 4 5 6 7 8 9 10 11
```

Rows are contiguous and adjacent. `grid[r][c]` is `*(int *)((char *)grid + r*16 + c*4)`. That is row-major order, and it is why iterating rows-then-columns is fast and columns-then-rows is slow — the cache behaviour you measured last week is a direct consequence of this layout, not a separate fact to memorise.

Note also what `grid[0]` decays to: `int *`, not `int **`. There is no array of row pointers anywhere in memory. An `int **` would be a genuinely different data structure with an extra indirection, and passing `grid` where `int **` is expected is a type error precisely because the memory layouts differ.

---

4. Lab A — read the machine's own answers

Open:

`03-c-toolchain-and-startup/challenges/day-001-pointer-and-array-lab.c`

Before compiling, write down your predictions for these eight questions. Do it on paper. Guessing and then reading is worth very little; committing and then being wrong is worth a lot.

1. What is `sizeof(char *)` compared with `sizeof(double *)`?
2. For a `struct sample { char tag; int value; char *name; }`, how many bytes does `p + 1` advance? (You will explain *why* tomorrow.)
3. Which of `arr`, `&arr[0]`, `&arr` print the same address, and which have different `sizeof`?
4. How far apart are `arr + 1` and `&arr + 1`?
5. What does `sizeof` report for an array parameter inside a callee?
6. What is the byte stride between `&grid[0][0]` and `&grid[1][0]`?
7. Which printed addresses change between two runs of the program, and which stay identical?
8. Is the address of a string literal near the stack or near the other constants?

Build and run:

```
cd 03-c-toolchain-and-startup/challenges
gcc -std=c17 -Wall -Wextra -O0 -g \
  day-001-pointer-and-array-lab.c -o /tmp/pointer-and-array-lab

/tmp/pointer-and-array-lab
/tmp/pointer-and-array-lab      # run it a second time
```

Question 7 is the one people get wrong. On this VM, `/proc/sys/kernel/ randomize_va_space` contains `2`, so the stack, heap, and shared-library mappings move on every execution while the static addresses of a non-PIE executable do not. Confirm it:

```
cat /proc/sys/kernel/randomize_va_space
setarch -R /tmp/pointer-and-array-lab | head -20
```

Under `setarch -R`, ASLR is disabled for that child only, and the stack addresses become reproducible. Use this whenever a lab needs the same address twice; do not disable ASLR system-wide.

A finding worth knowing about this machine

Section 7 of the lab prints one address from each storage region. On a default build here, you will see static objects at `0x40xxxx`, which tells you something specific: **the plain `gcc` on this system does not default to PIE.** Check it:

```
readelf -h /tmp/pointer-and-array-lab | grep Type
#   Type:                              EXEC (Executable file)

readelf -h /usr/bin/ls | grep Type
#   Type:                              DYN (Position-Independent Executable file)
```

Fedora's *packaged binaries* are hardened, because the distribution's build system injects `-fPIE -pie -fstack-protector-strong -D_FORTIFY_SOURCE=3 -Wl,-z,relro,-z,now` through its RPM build flags. The compiler itself, invoked directly on this VM, is configured upstream style and does none of that. You can ask for it explicitly:

```
gcc -std=c17 -Wall -Wextra -O2 -fPIE -pie -fstack-protector-strong \
    -D_FORTIFY_SOURCE=3 -Wl,-z,relro,-z,now \
    day-001-pointer-and-array-lab.c -o /tmp/hardened
readelf -h /tmp/hardened | grep Type
#   Type:                              DYN (Position-Independent Executable file)
```

Keep this distinction: **toolchain defaults** and **distribution policy** are different things. It will matter a great deal on Days 6 and 7, when the presence or absence of PIE and full RELRO changes what the PLT and GOT look like.

---

5. Strings: a data structure chosen in 1970 that you still pay for

C has no string type. It has a convention:

> A **string** is a contiguous sequence of characters terminated by the first null character. A "pointer to a string" is a pointer to its first character.

Nothing in the language enforces this. `char buf[8]` is eight bytes. Whether those bytes constitute a string depends entirely on whether a zero byte appears among them.

Why terminate rather than count?

The alternative was well known. Pascal-style strings put a length byte first:

```
length-prefixed:   [5]['h']['e']['l']['l']['o']
NUL-terminated:    ['h']['e']['l']['l']['o'][0]
```

Both use six bytes here. The trade-offs differ sharply:

```
                        length-prefixed        NUL-terminated
length of a string      O(1), read the byte    O(n), scan for zero
maximum length          255 with one byte      unbounded
a suffix of a string    needs a new header     just a pointer further in
copying                 length known up front  discovered while copying
appending               update the header      find the end first
```

That third row was the decisive one. In a NUL-terminated world, `s + 4` is a perfectly good string with no allocation and no copying. In a length-prefixed world it is not a string at all. On a machine where memory was the scarcest resource, being able to name substrings for free mattered.

The second row mattered too: a one-byte length caps you at 255 characters, and a larger length field costs space on every string in a 64 KiB address space.

Both choices were defensible in 1970. One of them turned out to make an entire class of security vulnerability structurally easy, because every string operation is a loop whose termination depends on data rather than on a count the caller supplied.

What the convention costs, concretely

```
size_t n = strlen(s);
```

is a loop. Calling it inside another loop turns an O(n) algorithm into O(n²), which is a real and common performance bug.

```
char dst[8];
strcpy(dst, src);
```

writes until it finds a zero in `src`. If `src` is longer than 7 characters, it writes past `dst`. `strcpy` has no parameter that could prevent this; the interface cannot be used safely without the caller already knowing the answer.

Worse, the failure is silent. Nothing returns an error. The write succeeds, into whatever happened to follow `dst`.

The functions that replaced them, and their exact contracts

Be precise about these, because their differences are the entire point.

`strncpy(dst, src, n)` — *not* a safe `strcpy`. It writes exactly `n` bytes: it pads with zeros if `src` is short, and it does **not** terminate if `src` is `n` or more characters. It was designed for fixed-width record fields in early Unix directory entries, not for strings. Using it as "safe strcpy" produces unterminated buffers.

`snprintf(dst, size, "%s", src)` — always terminates when `size > 0`. Returns the length it *would* have written, so `ret >= size` detects truncation. Available everywhere. This is the portable default.

`strlcpy(dst, src, size)` — always terminates, returns `strlen(src)`, so `ret >= size` detects truncation. Originally from OpenBSD; added to glibc in version 2.38, so it is available on this system's glibc 2.43, but it is not in ISO C.

`memcpy(dst, src, n)` — copies exactly `n` bytes and knows nothing about terminators. When you already know the length, this is the honest function to call, and it is the fastest.

You will implement the `strlcpy` contract yourself in Lab B. Implementing it is the fastest way to understand why "returns the source length" rather than "returns how much it copied" is the right design: the caller needs to know whether data was lost, and the amount copied cannot tell them that.

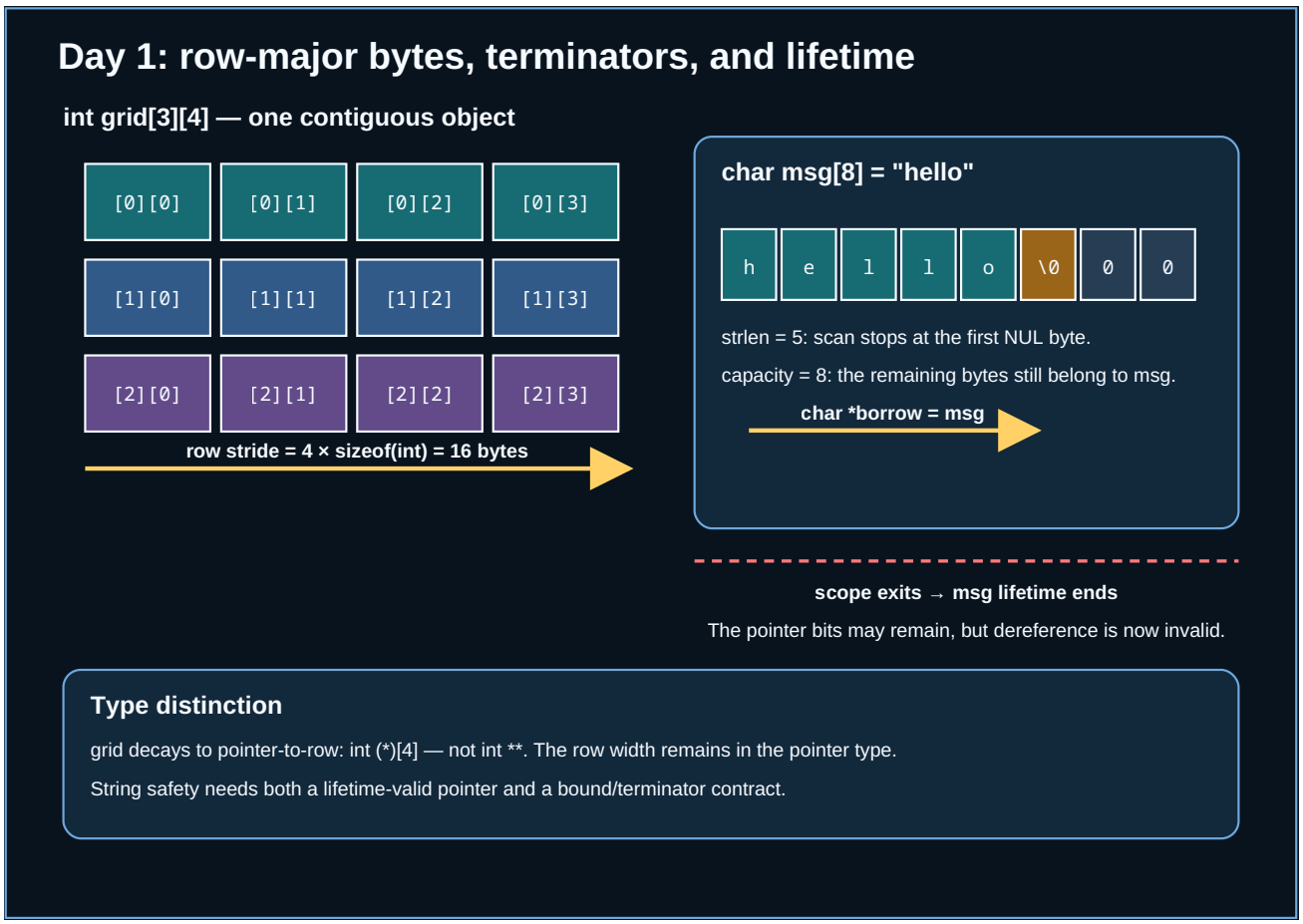
Literals are not arrays, even when they look like one

```
const char *literal  = "string literal";
char        writable[] = "string literal";
```

The first makes `literal` point into `.rodata`. The second creates a 15-byte array on the stack and copies the literal into it. Lab A shows both addresses and then writes to the array:

```
literal points at       0x401837
writable[] lives at     0x7ffca1ec4d01
after writable[0]='S':  "String literal"  (literal is still "string literal")
```

Writing through `literal` is undefined behaviour. On Linux it usually segfaults, because `.rodata` lands in a read-only mapping — which is a *loader* decision you will watch being made on Day 7, not a language guarantee. Declare literal pointers `const char *` so the compiler catches the mistake instead of the MMU.



Contiguous layout, a terminator convention, and object lifetime are separate parts of pointer validity.

---

6. Lab B — implement the primitives, then break them on purpose

Open:

`03-c-toolchain-and-startup/challenges/day-001-string-functions-lab.c`

Part one has five TODO functions: `my_strlen`, `my_strchr`, `my_strlcpy`, `my_strrev`, and `count_occurrences`. The tests are fixed; do not edit them.

```
gcc -std=c17 -Wall -Wextra -O0 -g \
  day-001-string-functions-lab.c -o /tmp/string-functions-lab
/tmp/string-functions-lab
```

Two of these have a subtlety worth pausing on before you start.

`my_strchr(s, '\0')` must find the terminator, not return `NULL`. Write the loop so that the comparison happens before the termination check, or you will fail that test and learn something useful about loop structure.

`my_strlcpy` returns `strlen(src)`, not the number of bytes copied. Read section 5 again if that seems backwards.

One deliberate compiler warning. This file is expected to produce exactly one diagnostic:

```
day-001-string-functions-lab.c:127:12: warning: function returns address
of local variable [-Wreturn-local-addr]
```

That is `bug3_dangling`, and it is there on purpose. Every other lab this week compiles silently under `-Wall -Wextra`.

Part two: making invisible bugs visible

The three `bug` modes are all wrong. Run each one in the plain build first and record what happens, because "nothing happened" is a result:

```
/tmp/string-functions-lab bug1
/tmp/string-functions-lab bug2
/tmp/string-functions-lab bug3
```

Then build with AddressSanitizer. **On this machine, gcc's ASan runtime package (`libasan`) is not installed**, so `gcc -fsanitize=address` fails at link time with `cannot find /usr/lib64/libasan.so.8.0.0`. Clang ships its own runtime and works out of the box:

```
clang -std=c17 -Wall -Wextra -O1 -g -fsanitize=address \
  day-001-string-functions-lab.c -o /tmp/string-functions-lab-asan

/tmp/string-functions-lab-asan bug1
```

```
==901644==ERROR: AddressSanitizer: heap-buffer-overflow on address
0x7bcaef3e0015 at pc 0x00000041ae3f
READ of size 6 at 0x7bcaef3e0015 thread T0
    #0 in strlen
    #1 in bug1_read_past_end day-001-string-functions-lab.c:110:9
0x7bcaef3e0015 is located 0 bytes after 5-byte region
  [0x7bcaef3e0010,0x7bcaef3e0015)
```

Read that carefully. `malloc(5)` gave five bytes, `memcpy` filled all five with characters, and `strlen` read a sixth looking for a terminator. The report names the exact byte, the exact source line, and the allocation it belongs to.

`bug2` reports a **stack-buffer-overflow** and even prints the frame layout showing which variable was overrun. `bug3` reports a **stack-use-after-scope** inside `printf`.

How AddressSanitizer knows

It is worth understanding, because it explains both its power and its limits. ASan does two things at compile time:

1. It replaces allocations with padded ones. Every heap block and every instrumented stack variable is surrounded by **redzones** — bytes marked poisoned.
2. It maintains **shadow memory**: one byte of metadata for every eight bytes of application memory, recording how many of those eight bytes are currently addressable.

Every load and store the compiler emits gets a shadow lookup inserted before it. If the shadow says poisoned, the program stops and prints the report. The cost is roughly 2× slower and 3× more memory, which is why this is a test-build tool, not a production one.

The limits follow from the mechanism. ASan finds *spatial* errors (reading outside an object) and *temporal* errors (reading an object after its lifetime). It does not find uninitialised reads — that is MemorySanitizer's job — and it does not find logic errors. Valgrind's Memcheck catches an overlapping but different set, works on unmodified binaries, and is far slower. You will use both this week.

---

7. The four ways a pointer goes wrong

Every pointer bug you will ever debug is one of these. Name them, so you can recognise them under time pressure.

Out of bounds

The pointer is valid, points at a real object, and you moved it too far. `bug1` and `bug2` are this. The fix is always a bound the code can actually check, which means the length has to be *present* — back to section 3.

Dangling

The pointer's bits are fine; the object they identify is gone. `bug3` returns the address of a local whose frame is about to be reused. The address is still a real, mapped address, which is why the program often does not crash. Tomorrow's lesson and Day 3 both return to this.

Uninitialised

```
int *p;
*p = 5;    /* p holds whatever was in that stack slot */
```

At `-O0` this often reads leftover stack garbage and segfaults. At `-O2` the compiler may reason that a correct program never does this and optimise on that basis. Day 4 shows exactly that happening.

Type-confused

The bits are a valid address, but you are reading the object through the wrong type. Sometimes this is deliberate and legal (`unsigned char` access is always allowed); sometimes it is a strict-aliasing violation with real consequences. Day 2 covers the legal forms and Day 4 covers what happens when you break the rule.

Where Linux draws a hard line

The kernel faces a fifth problem you never see in userspace: a pointer that a *user process* supplied. The kernel runs at a privilege level where dereferencing an arbitrary address would be catastrophic, and the address might be unmapped, might belong to a different process's page tables by the time it is used, or might point into kernel memory as a deliberate attack.

Kernel code therefore annotates such pointers:

```
ssize_t my_read(struct file *f, char __user *buf, size_t len, loff_t *off);
```

`__user` expands to nothing in a normal build, but the `sparse` static checker enforces that a `__user` pointer is never dereferenced directly. The only legal way to touch it is through the accessor functions:

```
if (copy_to_user(buf, kernel_data, len))
        return -EFAULT;
```

`copy_to_user` validates the range, handles a fault by returning the number of bytes not copied, and on modern x86-64 toggles SMAP — a CPU feature that traps kernel accesses to user pages unless explicitly permitted. This is the privilege boundary from Week 1 Day 7, now visible as a *pointer* discipline rather than as a mode bit.

Hold that thought. The same idea — "an address is only meaningful together with the address space it belongs to" — is what makes the rest of this course's virtual-memory and virtualization material work.

---

8. Predict, run, explain: a short integrated exercise

Do this on paper before touching a compiler.

```
char  msg[16] = "hello";
char *p = msg;
char *q = msg + 5;

/* 1 */  sizeof msg
/* 2 */  sizeof p
/* 3 */  strlen(msg)
/* 4 */  q - p
/* 5 */  *q
/* 6 */  msg[7]
/* 7 */  p[strlen(msg)]
/* 8 */  &msg[16]
/* 9 */  &msg[17]
```

For each one, state the **value** and the **C rule that produced it**. Items 6, 8, and 9 are the interesting ones:

- 6 is defined and predictable. `char msg[16] = "hello"` initialises all sixteen bytes; the initialiser supplies six and the rest are zeroed. That zero-fill rule applies to any array with an initialiser, and it is why `msg[7]` is 0 rather than garbage.
- 8 is the one-past-the-end address. Forming it is legal; dereferencing it is not.
- 9 is undefined behaviour at the moment the address is *formed*, before any dereference.

Then verify with a five-line program and explain any disagreement.

---

9. Mastery checkpoint

The tracker checkpoint for this section is:

> **Define an address precisely, explain pointer arithmetic, distinguish arrays from pointers, and handle strings safely.**

Explain

Without notes, and out loud:

- what a pointer value is at the machine level, and what its *type* adds that its bits do not;
- why `p + 1` advances by `sizeof(*p)`, and where that multiplication ends up in the generated code;
- what array-to-pointer decay is, which three contexts are exempt, and what information is destroyed by it;
- why `sizeof arr` and `sizeof p` differ even when `arr` and `p` hold the same address;
- why a NUL-terminated string makes `strlen` O(n) and substrings free, and which of those two the 1970s designers were buying;
- the exact contract of `strncpy`, `snprintf`, and `strlcpy`, and why only two of the three are safe;
- why the kernel refuses to dereference a `__user` pointer directly.

Draw

One diagram, on one page. Show a stack frame containing `int a[5]` and `int *p = &a[2]`. Mark:

- the five element boundaries with their byte offsets;
- the eight bytes that `p` itself occupies, and the arrow from them;
- where `a + 5` points and why that address is legal to form;
- the `.rodata` region, a string literal in it, and a `const char *` in the frame pointing at it;
- the direction the stack grows, and where the next call's frame would go.

Beside it, draw the same array as a `struct int_span` and mark what information the second version has that the first loses at a function boundary.

Observe

Run both labs and record:

1. Two runs of Lab A, plus one `setarch -R` run. Which addresses are stable and which are not, and what distinguishes the two groups?
2. `readelf -h` on your Lab A binary and on `/usr/bin/ls`. State the two `Type:` values and explain the difference in one sentence.
3. All three ASan reports from Lab B, with the specific byte offset each one names.

4. `objdump -d` your own `my_strlen` at `-O0` and at `-O2`. At `-O2`, gcc may vectorise it. Count the loads.

Build

1. Complete all five TODO functions in Lab B until every test passes.
2. Then change `my_strlcpy` to return the number of bytes it copied instead of `strlen(src)`, and write down which test breaks and what a caller could no longer detect.
3. Write a `struct int_span` version of a `sum` function and compare its call site with the `(pointer, count)` version. Which one can be called incorrectly, and how?

Why? notebook

1. Why does C pass arrays by reference-to-first-element rather than by value, and what would change if it did not?
2. Why is `&arr` a different type from `arr` when they print the same address?
3. Why is forming `a - 1` undefined even though the resulting address is usually mapped?
4. Why does `sizeof` not decay its array operand?
5. Why does a `char` array initialised from a shorter literal have defined values in its tail?
6. Why is `strncpy` not a safe `strcpy`, and what was it actually for?
7. Why can AddressSanitizer find a one-byte overflow that a normal run cannot?
8. Why does disabling ASLR make some lab addresses reproducible and leave others unchanged?
9. Why must the kernel treat a user-supplied pointer as data rather than as an address?

Mental model at the end

```
memory is a numbered array of bytes
        ↓
a load/store instruction takes one of those numbers
        ↓
C gives that number a TYPE, and calls it a pointer
        ↓ the type supplies the stride
pointer arithmetic moves in units of the pointed-to object
        ↓ objects laid end to end
an array is contiguous storage the compiler knows the size of
        ↓ decay, at almost every use
the callee receives only the first element's address
        ↓ the length must be carried some other way
counts, sentinels, or a pointer-plus-length structure
        ↓ the sentinel choice, for text
a string is "bytes until the first zero"
        ↓ therefore
every string operation is a loop bounded by data, not by the caller
```

The last arrow is the one worth keeping. C's string bugs are not carelessness; they are the predictable output of a representation whose end is discovered rather than declared. Once you see that, the correct habit follows on its own: *always know the length, and always know where it came from.*

References used selectively

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., §3.8 "Array Allocation and Access" and §3.9.1 "Structures", for the mapping from array indexing to x86-64 addressing modes. Local PDF: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`
- Kerrisk, *The Linux Programming Interface*, Ch. 6 "Processes", for the layout of text, data, bss, heap, and stack in a Linux process image — the regions Lab A prints addresses from. Local PDF: `references/library/books/02-os-and-linux/linux-programming-interface-michael-kerrisk.pdf`
- ISO/IEC 9899:2018 (C17), §6.3.2.1 for array-to-pointer conversion, §6.5.6 for additive operators and the one-past-the-end rule, and §6.5.2.1 for the definition of subscripting.
- Google's AddressSanitizer algorithm description, for shadow memory and redzones: https://github.com/google/sanitizers/wiki/AddressSanitizer
- Linux kernel documentation, `Documentation/dev-tools/sparse.rst` and the `__user` annotation, plus `arch/x86/include/asm/uaccess.h` for `copy_to_user` and the SMAP interaction: https://docs.kernel.org/dev-tools/sparse.html
- glibc 2.38 release notes for the addition of `strlcpy` / `strlcat`: https://sourceware.org/glibc/wiki/Release/2.38

After Day 1: bring your two lab transcripts, the one-page diagram, and your answers to the section 8 exercise. Tomorrow we group these objects together — and immediately discover that the compiler inserts bytes you did not ask for.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Pointer And Array Lab

Format: C lab

What you will do: Week 3, Day 1 — pointer and array observation lab.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-001-pointer-and-array-lab.c -lm -o /tmp/day-001-pointer-and-array-lab && /tmp/day-001-pointer-and-array-lab
```

- View lab source on GitHub

Put it into practice — String Functions Lab

Format: C lab

What you will do: Week 3, Day 1 — string primitives and explicit sanitizer experiments.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-001-string-functions-lab.c -lm -o /tmp/day-001-string-functions-lab && /tmp/day-001-string-functions-lab
```

- View lab source on GitHub

Day 2 — Structures, Unions, and Function Pointers

Tracker topic: Week 16 — Structures, Unions, and Function Pointers

Target time: approximately 2 hours 50 minutes **Rhythm:** 60 minutes layout and ABI · 35 minutes unions · 45 minutes callbacks · 30 minutes mastery

> A driver is not only data, and it is not only code. It is state plus a table of operations that another subsystem can call without knowing the concrete device. How can C express that contract?

Why this day exists

Programs quickly outgrew flat sequences of identical values. A process, packet, file, or device needs a record whose fields have different meanings. Compilers and ABIs also need an agreed physical layout so separately compiled code can exchange those records. Structures solve that problem.

Some storage must represent one of several alternatives. Unions provide overlapping member storage, but do not magically record which alternative is valid. Software must carry that invariant.

Finally, reusable systems code needs to invoke behavior selected at runtime. Function pointers and callback tables provide C's direct mechanism:

```
record related state
    → lay members out under alignment rules
    → represent a tagged alternative when needed
    → store addresses of compatible functions
    → select behavior through an operations table
```

Exact tracker objectives

- predict structure layout;
- use unions safely;
- understand callbacks;
- model kernel-style interfaces;
- cover structures and padding, union type-punning caveats, function pointers, callbacks, opaque handles, and intrusive lists;
- implement a callback registry, generic linked list, and miniature operations table.

Mastery checkpoint: explain how function-pointer tables enable polymorphism in C and in the Linux kernel.

---

1. Structure layout answers an ABI problem

```
struct record {
    char tag;
    uint32_t count;
    uint16_t flags;
};
```

Members appear in declaration order and have distinct storage. An implementation may insert unnamed padding between members and after the last member. It may not insert padding before the first member.

Why padding? A type has an **alignment requirement**. On a common x86-64 ABI, `uint32_t` wants an address divisible by 4. If `count` immediately followed the one-byte `tag`, it would begin at offset 1. The implementation commonly inserts three padding bytes:

```
offset  0       1 2 3       4 5 6 7       8 9       10 11
       +------+-------+----------------+----------+---------+
       | tag  | pad   | count          | flags    | tail pad|
       +------+-------+----------------+----------+---------+
```

Typical result for this ABI:

```
offsetof(tag)   = 0
offsetof(count) = 4
offsetof(flags) = 8
sizeof(record)  = 12
alignof(record) = 4
```

Tail padding makes every element of `struct record array[N]` correctly aligned:

```
address of array[i] = base + i × sizeof(struct record)
```

These numbers are observations for a target ABI, not universal C constants. Use `sizeof`, `_Alignof`/`alignof`, and `offsetof`; do not guess serialized formats from one compiler run.

Reordering can change size

Placing larger-alignment members first often reduces holes:

```
struct compact {
    uint32_t count;
    uint16_t flags;
    char tag;
};
```

But member order may be externally fixed by an ABI, protocol, hardware register layout, or file format. Smaller is not automatically more correct. Reordering public structures can break binary compatibility.

Padding bytes are not ordinary semantic fields

Whole-structure assignment copies the value as C defines it, but raw padding bytes can hold unspecified values. Comparing two structures with `memcmp` can report inequality even when all members compare equal. Writing a struct directly to disk or the network also bakes in:

- padding;
- endianness;
- integer widths;
- alignment/layout choices;
- possible uninitialized-byte disclosure.

Serialize fields explicitly into a defined byte format.

Packed is a contract with costs

Compiler attributes such as `__attribute__((packed))` are extensions, not a portable cure. Packed members can be misaligned. Some architectures trap; others perform slower accesses; compilers may need byte operations. Linux uses packed layouts when an external binary contract requires them, not as a default optimization.

---

2. Structures connect to machine addressing

For:

```
uint32_t get_count(const struct record *r) {
    return r->count;
}
```

`r->count` means `(*r).count`. The compiler knows `count`'s offset from the structure layout and can emit a load at:

```
address in r + offsetof(struct record, count)
```

The ABI is the bridge between translation units. If a shared library and an application disagree about packing options or structure definitions, the same member name can lead to different offsets and corrupted communication.

Linux exposes many internal structures but carefully versioned userspace interfaces should use fixed-width fields and explicit compatibility rules. Kernel layout is not a stable userspace ABI merely because headers can be read.

Flexible array members

A final incomplete array member can describe trailing storage:

```
struct message {
    size_t length;
    unsigned char payload[];
};
```

Allocate enough for header plus payload with overflow checks. `sizeof(struct message)` excludes payload elements and may include layout padding. This is not the historical “struct hack” of a one-element array.

---

3. Unions overlap storage

```
union payload {
    uint32_t number;
    float real;
    unsigned char bytes[4];
};
```

All members begin at the same address. The union is large and aligned enough for its largest/strictest member, subject to implementation rules. Only one member's value is stored at a time in the straightforward model.

The union does not remember which member is intended. Build a **tagged union**:

```
enum value_kind { VALUE_NUMBER, VALUE_REAL };

struct value {
    enum value_kind kind;
    union {
        uint32_t number;
        float real;
    } as;
};
```

Every write must update the member and tag consistently; every read must switch on the tag. The invariant is software policy.

Valid interpretation and type-punning caveats

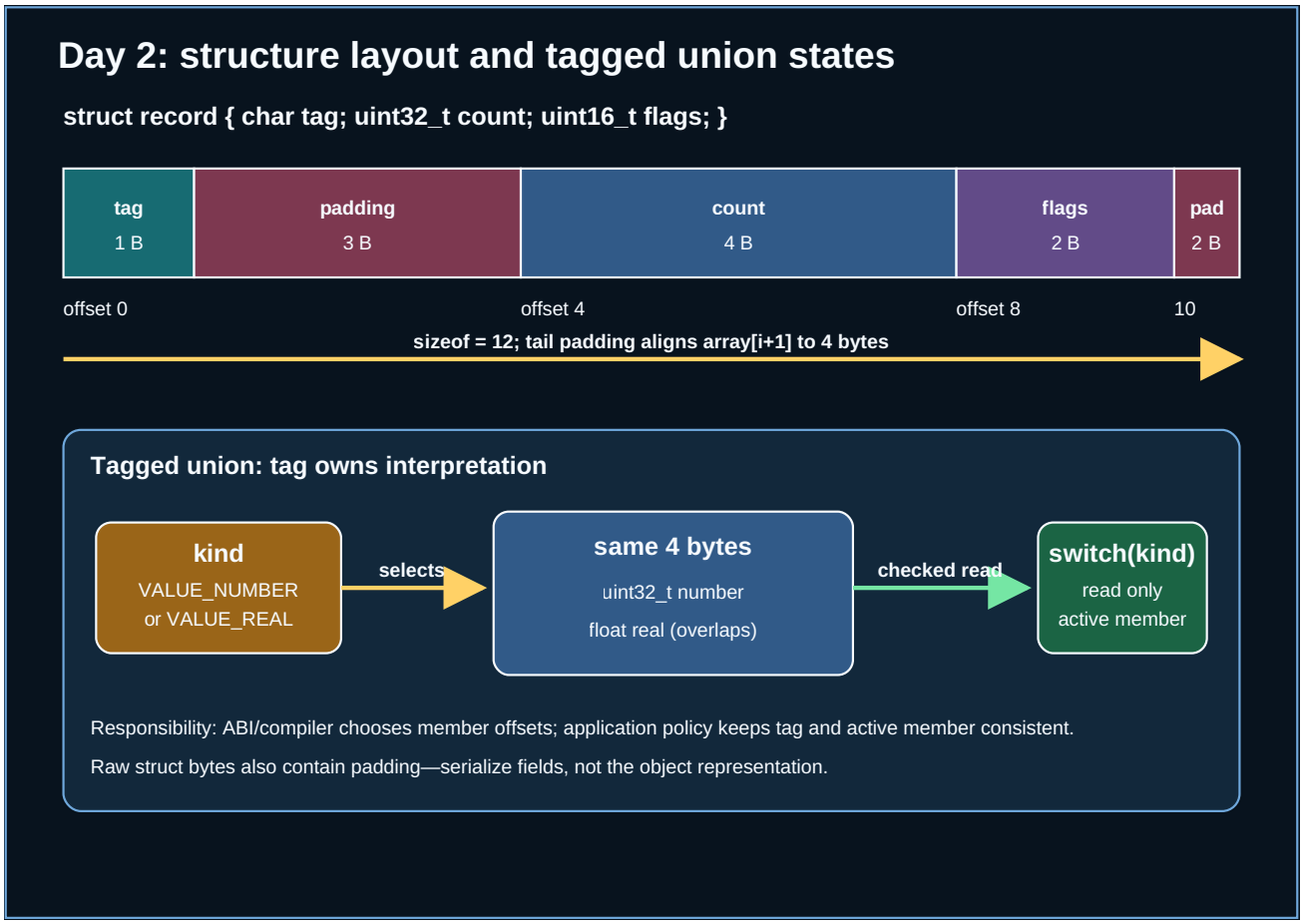
C permits carefully specified inspection of a union member other than the one most recently stored, with constraints and implementation-sensitive results for some representations. Common-initial-sequence rules allow limited inspection for compatible structure members inside a union. None of this means “unions make every cast safe.”

For converting object representations to bytes, character types have special permission to inspect representation. For bit-preserving conversion between unrelated scalar types, `memcpy` into a destination object is usually the clearest portable technique, while still requiring that the copied bits form a valid representation for that type.

Avoid teaching:

```
write any member → read any other member → portable reinterpretation
```

Also distinguish a union's overlapping storage from strict-aliasing rules applied through unrelated pointer types. A pointer cast does not create an object of the target type.



The ABI determines structure offsets; software must keep a union's tag consistent with its active member.

---

4. Function pointers are typed code-entry references

```
int add(int a, int b) { return a + b; }
int (*operation)(int, int) = add;
int result = operation(2, 3);
```

Read the declaration inside out: `operation` is a pointer to a function that takes two `int` arguments and returns `int`.

The function designator `add` converts to a pointer to the function in most expressions. Calling through the pointer requires a compatible function type. A cast can silence a diagnostic but cannot repair an incompatible calling sequence. Wrong parameter or return types create undefined behavior.

Use a typedef to make interfaces readable:

```
typedef int (*binary_operation)(int, int);
```

Object pointers and function pointers are distinct categories in portable C. POSIX `dlsym` lives at an implementation boundary with POSIX guarantees; do not generalize its idioms into ISO C.

Architecture lens — x86-64, AArch64, and RISC-V

Structure offsets and call sequences are target-ABI facts. The common x86-64 System V ABI, AAPCS64, and RISC-V psABI may give a simple structure the same layout, but their rules for alignment, aggregate arguments and returns, variadic calls, register ownership, and unwind state are not interchangeable.

For example, integer arguments begin in `RDI` on x86-64, `x0` on AArch64, and `a0` on RISC-V; return-address and stack conventions also differ. A compatible C function-pointer type expresses the language contract, while the compiler implements that call using the selected ABI. Never derive a wire format or cross-target callback interface from one native build. See the architecture comparison.

Callback: invert control deliberately

A callback API receives behavior to invoke later or for each item:

```
typedef void (*visitor_fn)(int value, void *context);

void visit(const int *items, size_t count,
           visitor_fn visitor, void *context);
```

The `void *context` carries caller-owned state without globals. Its lifetime must include every callback invocation. The callback's contract should state:

- when and how often it runs;
- whether it may unregister itself;
- thread/reentrancy rules;
- ownership of arguments;
- how errors or cancellation propagate.

Function type compatibility is only the first safety condition.

---

5. Operations tables: polymorphism made explicit

```
struct stream_ops {
    int (*read)(void *state, unsigned char *dst, size_t capacity);
    int (*close)(void *state);
};

struct stream {
    void *state;
    const struct stream_ops *ops;
};
```

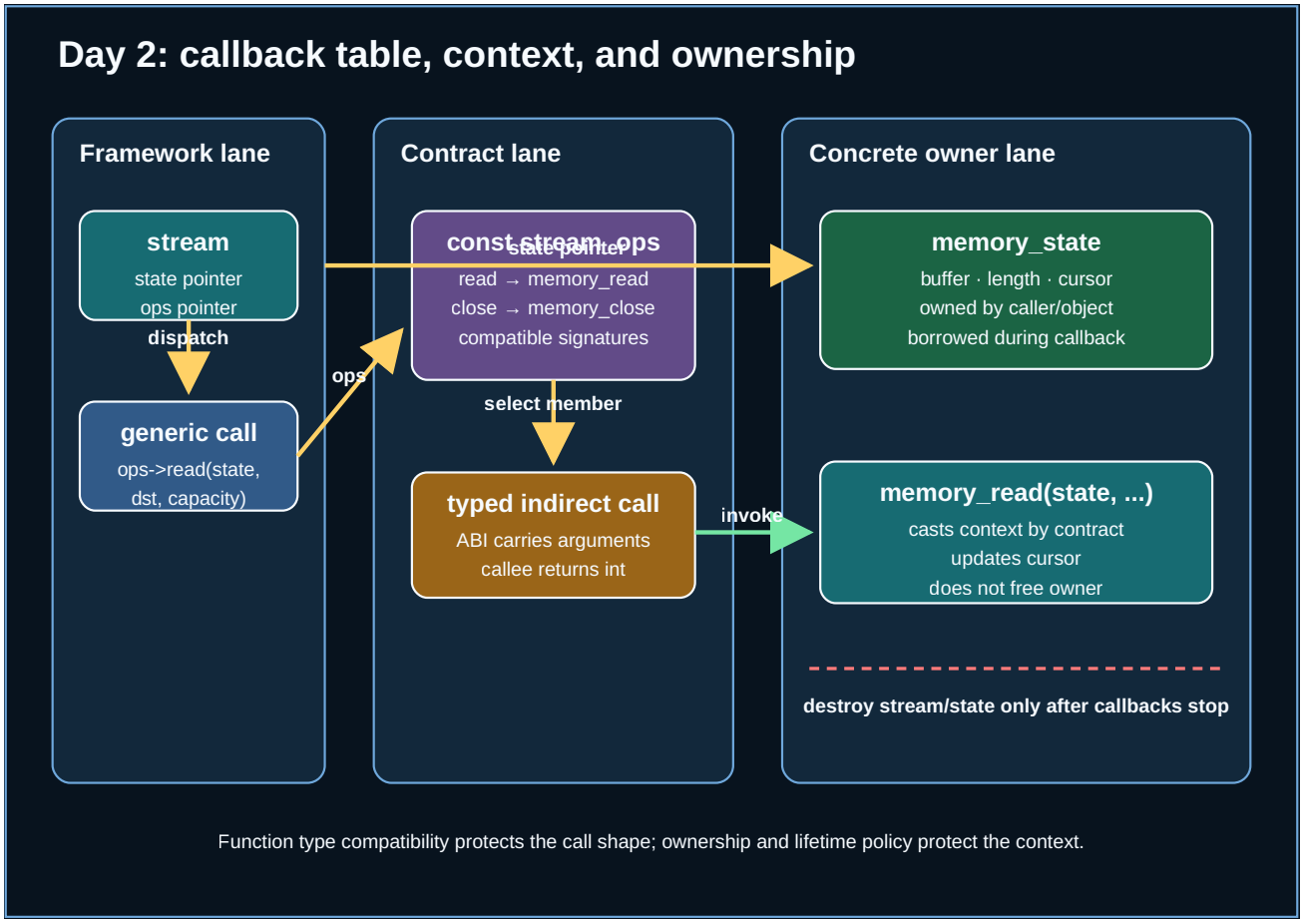
One concrete stream can use memory-backed functions; another can use file-backed functions. Generic code performs:

```
stream->ops->read(stream->state, buffer, sizeof buffer);
```

Trace the call:

```
stream object
  ├── state ──────────→ concrete private state
  └── ops ────────────→ immutable operation table
                          └── read function pointer
                                ↓ indirect call
```

This is runtime polymorphism without language-level classes. It works because each implementation honors the same function signatures and semantic contract.



Typed dispatch protects the call shape; the owner must keep callback context alive for every invocation.

Linux relies on this shape extensively: `struct file_operations`, network device operations, filesystem inode operations, clocks, buses, and many subsystems separate a common framework from implementation-specific behavior. The indirect call is only the mechanism; lifetime, locking, module ownership, and error contracts make it safe.

Opaque handles

A public header can declare:

```
struct registry;
struct registry *registry_create(void);
void registry_destroy(struct registry *);
```

Clients can carry the pointer but cannot access members because the structure definition remains private to the implementation. This reduces coupling and allows layout changes without recompiling code that only uses the API—subject to the library's ABI strategy.

Intrusive lists

An intrusive list embeds a linkage node inside the owned object:

```
struct task {
    int id;
    struct list_node link;
};
```

Given a `link` pointer, `container_of`-style logic recovers the enclosing `task` from the member offset. This avoids a separate wrapper allocation and supports one object in multiple lists through multiple link members.

The tradeoff is stronger coupling and stricter lifetime discipline. Removing the node does not necessarily destroy the enclosing object; freeing the object while linked leaves dangling list pointers.

---

6. Observe and build

Use `03-c-toolchain-and-startup/challenges/day-002-layout-callback-lab.c`.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g \
  03-c-toolchain-and-startup/challenges/day-002-layout-callback-lab.c \
  -o /tmp/week3-day2

/tmp/week3-day2 layout
/tmp/week3-day2 union
/tmp/week3-day2 callbacks
```

Before `layout`, draw offsets for all members and predict total size and alignment. Then compare with `offsetof`, `sizeof`, and `_Alignof`.

Inspect member addressing and indirect calls:

```
objdump -d -Mintel --disassemble=get_count /tmp/week3-day2
objdump -d -Mintel --disassemble=apply_operation /tmp/week3-day2
```

Find:

- the displacement matching `offsetof(count)`;
- the indirect call through a register or memory operand;
- the ABI registers carrying callback arguments.

Build tasks

1. Add one callback operation without changing `apply_operation`.
2. Add a fixed-capacity callback registry with duplicate-name rejection.
3. Complete the provided intrusive-list traversal and unlink one node.
4. Add a tagged-union string alternative with explicit ownership: either a borrowed immutable view or an owned allocation, but not an unnamed mix.

---

7. Mastery — Explain · Draw · Observe · Build

Explain

- Why can structure padding be needed between and after members?
- Why is `memcmp` not a general structure-value comparison?
- Why is a union insufficient without a tag for variant data?
- Why must a callback type and its semantic contract both match?
- How does an operations table enable C polymorphism?
- Why can changing a public structure layout break an ABI?

Draw

Draw one page containing:

1. a structure with offsets, alignment boundaries, and tail padding;
2. a tagged union with legal states;
3. object → ops table → selected function → concrete state;
4. an intrusive node embedded in two enclosing objects.

Observe

Record actual layout values, one member-load instruction, and one indirect call. State which facts come from ISO C and which come from this compiler/ABI.

Build

Demonstrate the registry and miniature operations table. Test:

- valid lookup;
- unknown operation;
- full registry;
- duplicate name;
- callback receiving caller context.

Why notebook

1. Why does an array of structures require tail padding?
2. Why should protocols not send raw in-memory structs?
3. Why is packed layout an external-contract tool rather than a default?
4. Why does a tagged union make invalid states easier to detect?
5. Why is casting an incompatible function pointer not a fix?
6. Why does callback context reduce global state?
7. Why are kernel operations tables more than “a list of addresses”?
8. Why does an intrusive list transfer lifetime responsibility to the owner?

References used selectively

- ISO/IEC 9899:2024 (C23), clauses 6.2.5, 6.2.6, 6.5, 6.7.3, 6.7.2.1 (structure/union specifiers), 6.7.7 (declarators), and 7.24.2 (`memcpy`): https://www.iso.org/standard/82075.html
- WG14 working draft N3220: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf
- System V AMD64 ABI, data representation and calling sequence: https://gitlab.com/x86-psABIs/x86-64-ABI
- GCC, “Common Type Attributes,” including `packed` and alignment extensions: https://gcc.gnu.org/onlinedocs/gcc/Common-Type-Attributes.html
- Linux kernel documentation, “Data structures and low-level utilities,” linked lists and `container_of`: https://docs.kernel.org/core-api/kernel-api.html
- Linux kernel source, `include/linux/fs.h`, `struct file_operations`, as a current operations-table example: https://github.com/torvalds/linux/blob/master/include/linux/fs.h
- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 3 §§3.10–3.11 on heterogeneous data structures and alignment. Local catalog path: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`

Next bridge: a record can point to state, but who owns that state, when does its lifetime begin, and which path releases it? Day 3 makes allocation and cleanup contracts explicit.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Layout Callback Lab

Format: C lab

What you will do: Week 3, Day 2 — structures, tagged unions, callbacks, and intrusive links.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-002-layout-callback-lab.c -lm -o /tmp/day-002-layout-callback-lab && /tmp/day-002-layout-callback-lab layout
```

- View lab source on GitHub

Day 3 — Dynamic Memory and Lifetime

Tracker topic: Week 17 — Dynamic Memory and Lifetime

Target time: approximately 3 hours **Rhythm:** 55 minutes lifetime model · 45 minutes allocation interfaces · 55 minutes arena/diagnostics · 25 minutes mastery

> A pointer can still contain the same address bits after `free`. Why does an access that worked one instruction earlier become invalid?

Why this day exists

Automatic local storage is excellent when lifetime follows a function call. Real systems also create objects whose number and lifetime depend on input: connections, syntax trees, packets, caches, and jobs. Their storage must outlive one scope and eventually be reclaimed.

C separates mechanism from policy. `malloc` supplies storage; it does not know the intended type, owner, graph, or cleanup time. This flexibility made C practical for operating systems and runtimes, and it makes ownership bugs possible.

```
program needs runtime-sized or independently lived object
    → allocator reserves suitably aligned storage
    → program establishes object state
    → ownership rules govern aliases and transfer
    → last required use ends
    → cleanup releases resources exactly once
```

Exact tracker objectives

- understand heap allocation interfaces;
- distinguish ownership from reachability;
- recognize leaks, double frees, and use-after-free;
- design cleanup paths;
- cover `malloc`, `calloc`, `realloc`, `free`, allocator metadata, ownership conventions, RAII alternatives in C, and error unwinding;
- implement a simple arena allocator;
- run Valgrind or sanitizers;
- refactor code to have explicit ownership rules.

Mastery checkpoint: for every allocated object, identify its owner, lifetime, and cleanup path.

---

1. Storage duration is not “where bytes look like they are”

C defines storage duration—the minimum potential lifetime of storage—rather than a universal stack/heap architecture:

- **automatic:** commonly block locals, usually lasting through block execution;
- **static:** lasts for the entire program execution;
- **thread:** one instance per thread for the thread's execution;
- **allocated:** obtained from an allocation function until deallocated.

Function-local automatic objects are commonly placed in a machine stack frame, but the compiler may keep them only in registers, optimize them away, or reuse stack slots. Variable-length arrays, `alloca` extensions, coroutine runtimes, split stacks, and escape analysis make slogans unreliable.

Likewise, “the heap grows upward” is not a C guarantee or a complete Linux model. A libc allocator may obtain virtual memory through `brk`, `mmap`, or other mechanisms; maintain arenas, bins, caches, and metadata; and return blocks from previously acquired regions.

Use precise layers:

```
C allocated storage
    ↓ implemented by
libc allocator policy and metadata
    ↓ requests/mappings through
Linux virtual-memory interfaces
    ↓ backed lazily by
physical pages / swap / file mappings
```

Lifetime and scope differ

Scope controls where an identifier is visible in source. **Lifetime** controls when an object exists. A pointer returned from a function can be in scope for the caller while its pointee is already dead:

```
int *bad(void) {
    int local = 42;
    return &local;       /* local lifetime ends on return */
}
```

The address bits might still name mapped stack memory. That does not extend the object's lifetime.

---

2. Ownership, aliasing, and reachability

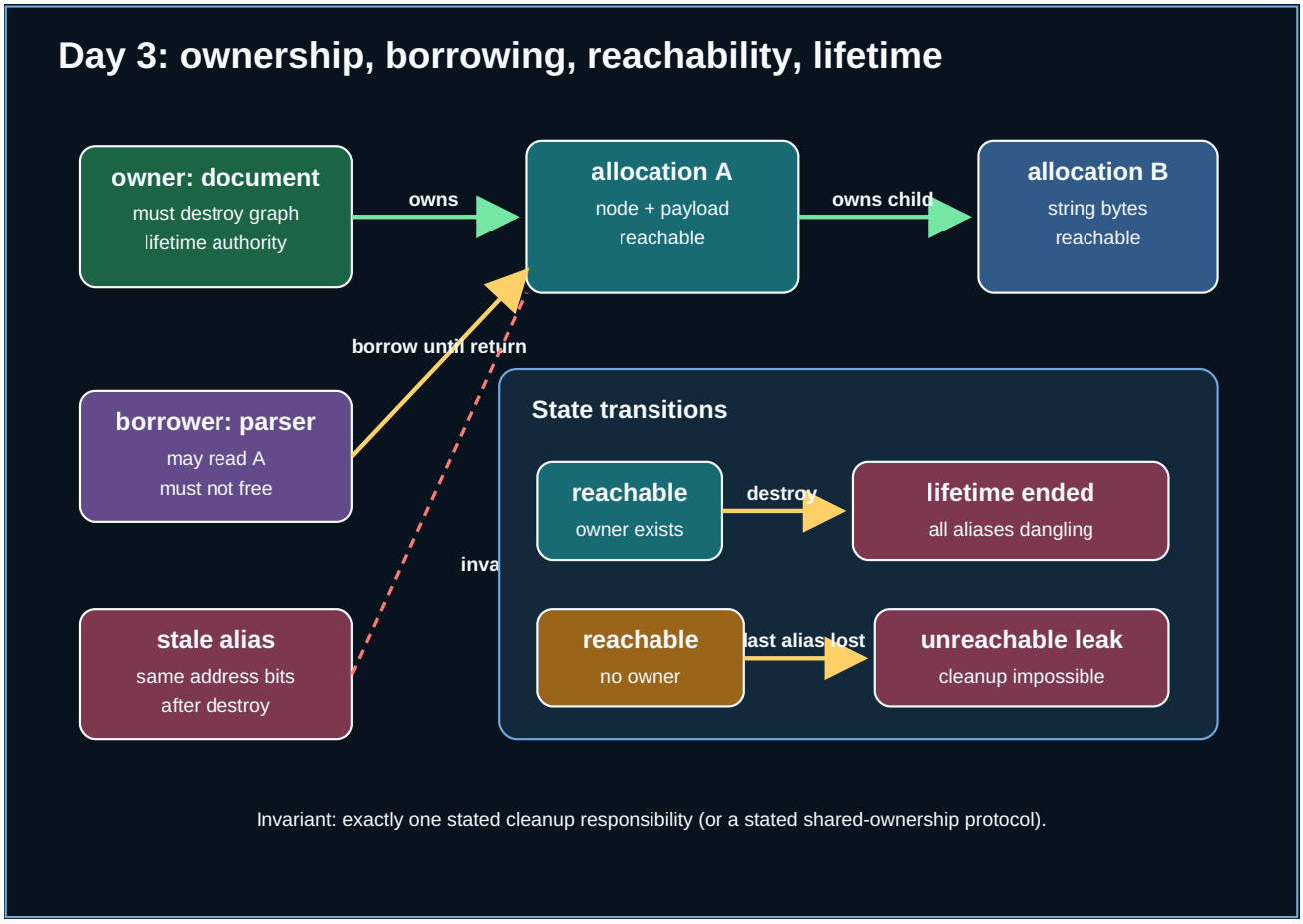
C has no built-in ownership checker, so APIs need conventions.

- An **owner** is responsible for releasing a resource.
- A **borrower** may use it within an agreed interval but must not release or retain it beyond that interval.
- An **alias** is another access path to the same object.

- **Reachability** means some pointer path can still name the allocation.

Ownership and reachability are different:

```
reachable but no owner     → eventual leak or ambiguous cleanup
owned but stale alias      → use-after-free risk
unreachable allocation     → definite leak
two claimed owners         → double-free risk
```



Reachability answers whether an address can be found; ownership answers who must end the lifetime.

A linked graph can be fully reachable but have no documented component responsible for destruction. A reference-counted object can have many aliases and a precise shared-ownership rule. A cache may intentionally keep reachable objects until shutdown.

Write contracts in API names/comments:

```
create/open/new      returns owned resource
borrow/view/get      returns non-owning access unless documented otherwise
take/adopt           transfers ownership in
release/destroy/free consumes ownership
clone/dup            creates another owned resource
```

Names are conventions, not language enforcement. Tests and review must verify them.

---

3. The allocation family

`malloc`

```
int *values = malloc(count * sizeof *values);
```

On success it returns suitably aligned storage of the requested size. The bytes have indeterminate values. On failure it returns null.

Before multiplication, check overflow:

```
if (count > SIZE_MAX / sizeof *values)
    return NULL;
```

Prefer `sizeof *values`; it follows the pointed-to type if the declaration changes.

For a zero-size request, C permits behavior that requires care; do not build an ownership protocol around dereferencing a result or assuming one universal non-null policy. Normalize zero counts explicitly where useful.

`calloc`

```
int *values = calloc(count, sizeof *values);
```

It checks the conceptual element multiplication according to its interface and initializes all bytes to zero. All-bits-zero represents integer zero, but do not generalize that representation to every possible pointer or floating-point representation in every C implementation.

`realloc`

`realloc` can resize allocated storage, perhaps moving it. The safest common shape is:

```
void *temporary = realloc(buffer, new_size);
if (temporary == NULL) {
    /* old buffer remains owned and valid */
    handle_failure();
} else {
    buffer = temporary;
}
```

Do not assign directly to the sole owner:

```
buffer = realloc(buffer, new_size); /* leaks old block if failure returns NULL */
```

On success, old pointers into the prior allocation become invalid even if the numeric base address happens not to change; use the returned pointer. Bytes up to the smaller old/new size are preserved. Newly added bytes are not initialized.

`free`

`free(NULL)` is a no-op. Otherwise, pass exactly a pointer value accepted by the deallocation contract: normally the current base pointer returned by the allocation family and not already freed.

After `free(p)`:

- the allocation's lifetime ends;
- aliases become dangling;
- reading, writing, or freeing through them is invalid;
- assigning `p = NULL` protects only that one variable, not other aliases.

The allocator may retain the physical pages or block for later reuse. “Free” means the program relinquished the allocation; it does not promise immediate zeroing or return to the OS.

---

4. Four failure classes

Leak

An allocation remains unreleased after the program no longer needs it. Long-running services turn small leaks into resource exhaustion.

Use-after-free

An access occurs after lifetime ended. Reuse can make the stale pointer read another object's data or corrupt allocator metadata.

Double free

The same allocation is deallocated twice without an intervening new valid ownership event. Allocator checks may abort, but the language does not promise a friendly diagnostic.

Interior/foreign free

Calling `free(p + 1)`, freeing automatic storage, or freeing a pointer from a different allocation API violates the deallocator's contract.

Sanitizers and Valgrind improve detection. They do not make invalid programs defined.

---

5. Allocator concepts without pretending to implement libc

A general allocator tracks free and used regions. Common design ideas include:

- block metadata, either adjacent to user storage or out-of-band;
- size classes/bins for quick reuse;
- splitting a large free block;
- coalescing adjacent free blocks;
- per-thread caches and multiple arenas for concurrency;
- alignment and minimum block sizes;
- fragmentation—free bytes exist but not in a useful shape;
- requesting/releasing larger virtual-memory regions from/to the kernel.

The pointer returned to the caller usually addresses usable storage, not the allocator's complete block. Writing before or after it can corrupt metadata or neighboring objects.

Linux's userspace allocator is not the kernel page allocator. Inside the kernel, APIs such as `kmalloc`, slab caches, and page allocation have distinct contexts and constraints. User code must not infer kernel allocation behavior from glibc internals.

Arena allocation

An arena obtains one backing region and serves aligned slices by bumping an offset:

```
base
  ↓
 +----------+---------+-------------+----------------------+
 | object A | padding | object B    | unused capacity      |
 +----------+---------+-------------+----------------------+
                         ↑ next offset
```

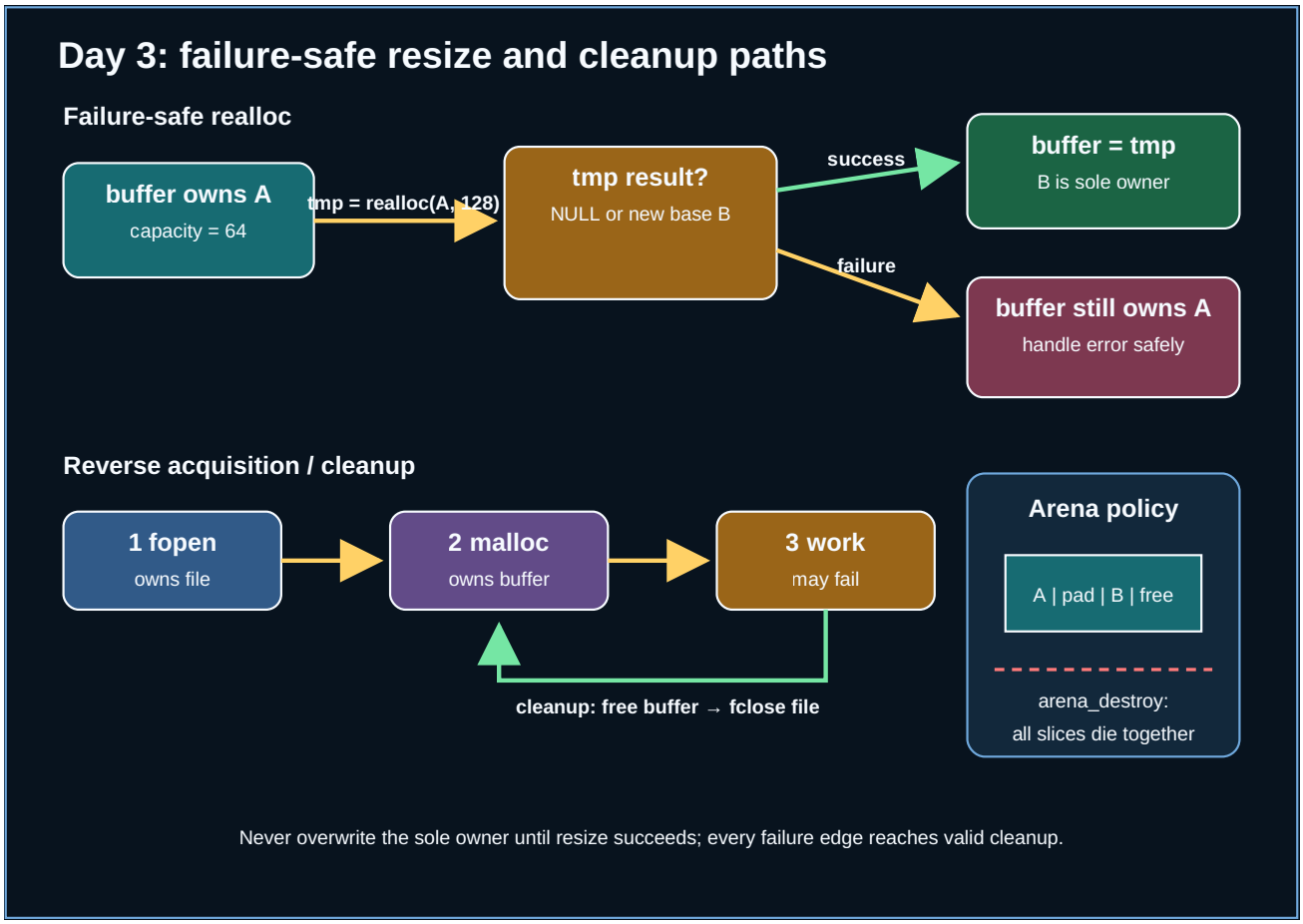
Advantages:

- very cheap allocation;
- no per-object free;
- one bulk cleanup;
- clear “all objects die together” lifetime.

Tradeoffs:

- cannot reclaim arbitrary individual objects;
- arena lifetime must dominate every borrowed pointer;
- large/long-lived object mixtures can waste storage;
- alignment and overflow must be correct.

This is an ownership strategy, not a replacement for every allocator.



Preserve the old owner until resize succeeds, and make every failure edge reach a valid cleanup policy.

---

6. Cleanup paths and RAII alternatives in C

C does not have standard C++-style destructors, but cleanup can be systematic.

Single-exit cleanup ladder

```
int process(void)
{
    int result = -1;
    FILE *file = NULL;
    unsigned char *buffer = NULL;

    file = fopen("input", "rb");
    if (file == NULL)
        goto out;

    buffer = malloc(4096);
    if (buffer == NULL)
        goto out;

    /* work */
    result = 0;

out:
    free(buffer);
    if (file != NULL)
        fclose(file);
    return result;
}
```

`goto` here expresses error unwinding, not arbitrary control flow. Every resource is initialized to an empty state; cleanup runs in reverse acquisition order; cleanup operations tolerate absence when possible.

Other patterns include:

- dedicated `*_init`/`*_destroy` pairs;
- owner structs collecting related resources;
- GCC/Clang cleanup attributes as nonportable extensions;
- scope macros, with care about control flow;
- arena/region lifetime;
- reference counting;
- generated defer mechanisms.

The standard-C baseline remains explicit cleanup.

Design the cleanup before the happy path

For each acquisition, ask:

```
Who owns it after success?
What happens if the next operation fails?
Can cleanup itself fail?
In what reverse order must resources be released?
Which aliases must stop being used?
```

This is especially important in kernel code, where locks, references, DMA mappings, and device resources have strict context and order requirements.

---

7. Observe and build

Use:

- `03-c-toolchain-and-startup/challenges/day-003-arena-lab.c`
- `03-c-toolchain-and-startup/challenges/day-003-memory-errors.c`

Build the safe arena:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g \
  03-c-toolchain-and-startup/challenges/day-003-arena-lab.c \
  -o /tmp/week3-day3-arena

/tmp/week3-day3-arena
```

Predict offsets and alignment before running. Verify that out-of-capacity allocation returns null without changing arena state.

Build the diagnostic fixture:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O1 -g \
  -fsanitize=address,undefined -fno-omit-frame-pointer \
  03-c-toolchain-and-startup/challenges/day-003-memory-errors.c \
  -o /tmp/week3-day3-errors

/tmp/week3-day3-errors safe
/tmp/week3-day3-errors leak
/tmp/week3-day3-errors uaf
/tmp/week3-day3-errors double-free
```

Each invalid mode runs only when explicitly selected. Expect nonzero exits for detected UAF/double-free. Do not automate invalid modes in a script with `set -e` unless you deliberately capture their expected failures.

If Valgrind is installed, build without ASan and run one mode at a time:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
  03-c-toolchain-and-startup/challenges/day-003-memory-errors.c \
  -o /tmp/week3-day3-valgrind

valgrind --leak-check=full --show-leak-kinds=all \
  /tmp/week3-day3-valgrind leak
```

Do not combine Valgrind and ASan in one run; they instrument memory through different runtimes and are best used separately.

Refactor task

Take this vague interface:

```
char *load_text(const char *path);
```

Write a complete contract:

- ownership of the returned pointer;
- how length is returned;

- termination guarantee;
- empty file behavior;
- allocation-failure behavior;
- maximum/overflow policy;
- which function releases it.

Then implement it with one cleanup path and tests for missing, empty, and small files.

---

8. Mastery — Explain · Draw · Observe · Build

Explain

- Why are scope, storage duration, and lifetime different?
- Why is allocated storage not simply “RAM on the heap”?
- Why does reachability not prove ownership?
- Why must `realloc` use a temporary owner?
- Why can `free` finish while RSS does not immediately fall?
- Why does nulling one pointer not repair stale aliases?

Draw

Draw:

1. pointer aliases around one allocation, marking owner and borrowers;
2. an error-unwinding ladder in reverse acquisition order;
3. libc allocator → Linux mappings → pages;
4. arena base, aligned objects, offset, and bulk lifetime boundary.

Observe

Save the arena output, one ASan report, and one leak report. Annotate:

- allocation site;
- lifetime-ending event;
- invalid access or missing cleanup;
- ownership rule that would prevent it.

Build

Complete the arena and `load_text` contract. For every allocated object in your implementation, write:

```
object | owner | borrowers | lifetime end | cleanup path | failure path
```

Why notebook

1. Why does dynamic size require a lifetime policy, not merely `malloc`?
2. Why can a stale pointer retain plausible address bits?
3. Why is allocator metadata vulnerable to out-of-bounds writes?

4. Why is `calloc` not just a spelling convenience for every type?
5. Why is individual `free` absent from a bump arena?
6. Why should cleanup proceed in reverse acquisition order?
7. Why can sanitizers miss a bug in a particular execution?
8. Why do kernel allocators have context-specific APIs?

References used selectively

- ISO/IEC 9899:2024 (C23), clauses 6.2.4 (storage durations), 7.24.3 (memory management functions), and 7.24.2 (`memcpy`): https://www.iso.org/standard/82075.html
- WG14 working draft N3220: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf
- Linux man-pages, `malloc(3)`, `free(3)`, `realloc(3)`, `calloc(3)`, `brk(2)`, and `mmap(2)`: https://man7.org/linux/man-pages/man3/malloc.3.html
- GCC and Clang sanitizer documentation: https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html https://clang.llvm.org/docs/AddressSanitizer.html
- Valgrind Memcheck manual: https://valgrind.org/docs/manual/mc-manual.html
- Linux kernel documentation, memory allocation guide: https://docs.kernel.org/core-api/memory-allocation.html
- Kaiwan N. Billimoria, *Hands-On System Programming with Linux* (2018), chapters on process virtual address space, dynamic memory, and memory debugging. Local catalog path: `references/library/books/02-os-and-linux/hands-on-system-programming-with-linux-kaiwan-billimoria-2018.pdf`
- Kaiwan N. Billimoria, legacy memory-management training, Part 2, conceptual process mappings only; implementation details require current verification: `references/library/training/kaiwantech/memory-management-legacy/02-kernel-and-process-memory-segments.pdf`

Next bridge: lifetime rules tell us when an object may be accessed. Qualifiers, atomics, and the abstract machine determine what the compiler may assume about those accesses—especially when devices or other threads are involved.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Arena Lab

Format: C lab

What you will do: Day 3 arena lab: a small, defined bump allocator.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-003-arena-lab.c -lm -o /tmp/day-003-arena-lab && /tmp/day-003-arena-lab
```

- View lab source on GitHub

Put it into practice — Memory Errors

Format: C lab

What you will do: Day 3 memory diagnostics fixture.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-003-memory-errors.c -lm -o /tmp/day-003-memory-errors && /tmp/day-003-memory-errors safe
```

- View lab source on GitHub

Day 4 — Qualifiers, Atomics, and Undefined Behavior

Tracker topic: Week 18 — Qualifiers, Atomics, and Undefined Behavior

Target time: approximately 2–3 hours **Rhythm:** 40 minutes qualifiers · 45 minutes behavior/optimization · 45 minutes atomics · 35–50 minutes labs and mastery

> If the CPU can execute a load, why may the compiler remove it? If two CPU cores can both see one address, why is an ordinary C data race undefined?

Why this day exists

Day 3 established that an object may be accessed only during its lifetime. That is necessary, but not sufficient. The compiler also reasons about the object's type, qualifiers, aliases, side effects, and cross-thread ordering.

C is not portable assembly. It specifies an **abstract machine** and permits any translation that preserves the observable behavior of a program whose behavior is defined. This freedom enables optimization. It also means that a source operation cannot be used for a job its language semantics do not perform.

```
source-level contract
  ├─ lifetime and type rules
  ├─ qualifier promises
  ├─ sequencing and observable side effects
  └─ atomic synchronization
          ↓ compiler maps the contract
target ISA instructions
          ↓ architecture-specific memory system
caches, interconnect, devices, and cores
```

The layers connect, but they are not interchangeable. A strong hardware memory model does not define a C data race. A `volatile` load is not an atomic operation. One instruction on one target is not a portable C guarantee.

Tracker contract

By the end, you should be able to:

- state the precise roles of `const`, `volatile`, and `restrict`;
- explain why `volatile` is not thread synchronization;
- distinguish undefined, unspecified, and implementation-defined behavior;
- connect observable behavior to compiler optimization;
- recognize signed-overflow, aliasing, sequencing, race, and lifetime UB;
- use C atomics for a relaxed counter and release/acquire publication;
- distinguish the C memory model from an ISA or device-memory model.

Mastery checkpoint: explain how the compiler's freedom under the C abstract machine affects systems code.

---

1. Qualifiers are contracts, not decorations

`const`: no modification through this access path

```
int checksum(const unsigned char *data, size_t length);
```

Here `const` says the function does not modify the bytes through `data`. It improves an API contract and lets the function accept genuinely nonmodifiable objects. It does **not** necessarily mean:

- the object lives in read-only physical memory;
- no other non-const alias can modify the object;
- the value is a translation-time constant;
- a load must happen every time the identifier appears.

```
int value = 1;
const int *view = &value;
value = 2;                  /* valid: value itself was not defined const */
printf("%d\n", *view);      /* observes 2 */
```

By contrast, casting away `const` and modifying an object that was defined with a const-qualified type has undefined behavior:

```
const int fixed = 1;
int *forced = (int *)&fixed;
*forced = 2;                /* undefined behavior */
```

Read declarations from the identifier outward:

```
const int *p;        /* pointer to const int */
int *const p2 = ...; /* const pointer to int */
```

`volatile`: accesses are externally significant

A volatile-qualified object tells the implementation that accesses are observable side effects and must be evaluated according to the abstract machine's volatile rules. Typical legitimate uses include:

- device registers in a platform where the implementation and device API define volatile access as appropriate;
- objects changed by a signal handler, using the narrow types and operations the signal rules permit;
- low-level implementation interfaces that explicitly require it.

It does not provide:

- atomicity for a multi-byte or read-modify-write operation;
- mutual exclusion;

- a cross-thread happens-before relation;
- a portable compiler or CPU memory fence;
- cache coherence beyond what the platform already defines.

```
volatile int ready;
int payload;

/* Thread A */ payload = 42; ready = 1;
/* Thread B */ while (!ready) {} printf("%d\n", payload);
```

This is not a C synchronization protocol. The volatile accesses remain volatile, but the ordinary conflicting cross-thread accesses have no atomic synchronization. If they form a data race, behavior is undefined.

For memory-mapped I/O, `volatile` may be one ingredient, but device ordering, access width, barriers, endianness, and mapping attributes come from the platform. Linux kernel helpers such as `readl()`/`writel()` embody kernel and architecture contracts; they are not merely ISO C volatile expressions.

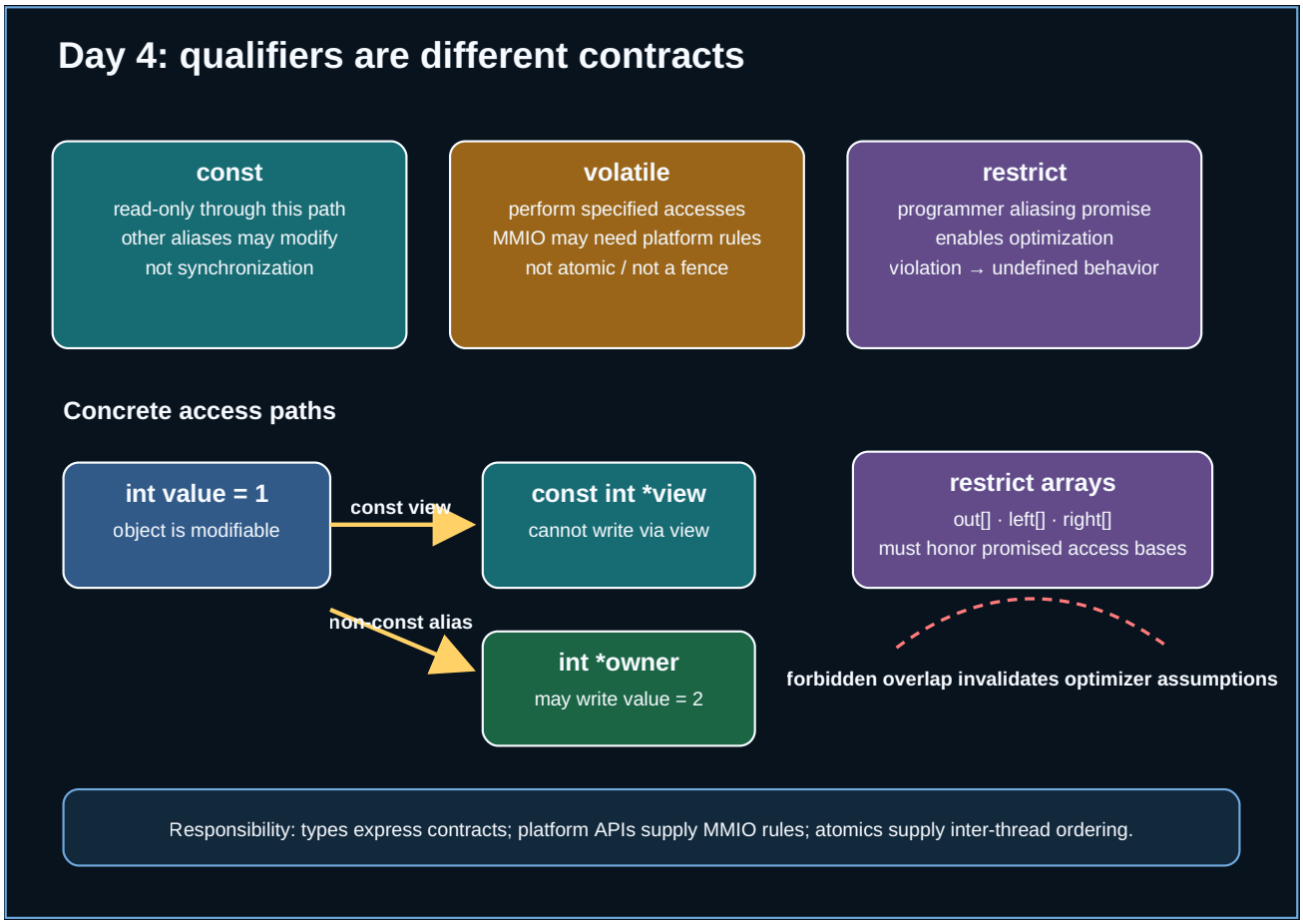
`restrict`: an aliasing promise made by the programmer

For a pointer used to access an object during a block execution, `restrict` can promise that relevant accesses to that object are based on the designated pointer rather than competing independent access paths. That lets a compiler avoid repeatedly assuming that stores through one parameter change values read through another.

```
void add(size_t n, int *restrict out,
         const int *restrict left, const int *restrict right)
{
    for (size_t i = 0; i < n; ++i)
        out[i] = left[i] + right[i];
}
```

Calling this with appropriately separate arrays honors the contract. Calling it so that the restricted access paths overlap in a way forbidden by the contract produces undefined behavior. `restrict` does not check pointers at runtime and does not magically make them non-overlapping.

The exact standard wording is more nuanced than “these pointers never alias.” Use that slogan only as an initial intuition; reason about which objects are accessed, whether they are modified, and which pointer expressions those accesses are based on.



The three qualifiers constrain different actors and do not substitute for one another.

---

2. Four behavior categories

These labels answer different specification questions.

Defined behavior

The standard imposes requirements. Unsigned arithmetic is reduced modulo one more than the type's maximum value. A conforming implementation must preserve that result.

Implementation-defined behavior

The implementation chooses one allowed behavior and documents the choice. Examples include whether plain `char` has the same range as `signed char` or `unsigned char`, and many properties of integer types.

This is not automatically erroneous. It is a portability dependency to find, document, or avoid.

Unspecified behavior

The standard allows two or more possibilities and does not require the implementation to document which occurs in a particular instance. One common example is which function argument is evaluated first when no rule imposes an order.

Unspecified does not mean “anything can happen.” The result must still be one of the permitted possibilities, assuming no separate undefined behavior is triggered.

Undefined behavior (UB)

The standard imposes no requirements after the program executes an undefined operation. Examples relevant today include:

- signed integer overflow;
- an out-of-bounds access;
- use after an object's lifetime ends;
- a forbidden access through an incompatible type;
- violating an applicable `restrict` association;
- unsequenced conflicting side effects, such as `i = i++ + 1`;
- a data race between threads.

UB is not a predictable fallback result, a guaranteed crash, or necessarily a hardware trap. A test may appear to work until optimization, input, compiler, or surrounding code changes.

A compact classification drill

Situation	Category
`UINT_MAX + 1U`	defined modulo arithmetic
whether plain `char` is signed	implementation-defined
order of many argument evaluations	unspecified
`INT_MAX + 1` as `int`	undefined
read after `free`	undefined
racing ordinary reads/writes	undefined

The classification belongs to the C specification. The CPU may use the same addition instruction for a defined unsigned addition and an undefined signed overflow.

---

3. Observable behavior and optimization

The compiler need not preserve source text step for step. It must preserve the required observable behavior of a defined program. Depending on the C version and execution environment, that includes specified interactions such as volatile accesses, file/stream effects, and atomic operations.

```
int square_and_discard(int x)
{
    int result = x * x;
    return 0;
}
```

This is not harmless for every `x`: the signed multiplication can overflow. If the compiler proves the result is unused, it may remove the calculation, and then no overflowing evaluation occurs at runtime. Source appearance alone does not prove an operation survives translation.

Now consider:

```
int greater_after_increment(int x)
{
    return x + 1 > x;
}
```

For every execution in which `x + 1` is defined, the comparison is true. The compiler may return true directly. “But the machine instruction wraps at `INT_MAX`” applies a CPU fact where the C contract gave no wrapping result. Use an unsigned type or an explicit checked operation when wrapping or overflow detection is the intent.

Aliasing example

Accessing stored data through an incompatible lvalue type can violate C's aliasing/type-access rules:

```
float value = 1.0f;
unsigned int bits = *(unsigned int *)&value; /* not a portable bit copy */
```

Use `memcpy` to copy an object representation into a same-sized destination, or use character-type access to inspect representation bytes:

```
uint32_t bits;
_Static_assert(sizeof bits == sizeof value, "lab assumption");
memcpy(&bits, &value, sizeof bits);
```

This tells the optimizer the real operation instead of hiding it behind an invalid type claim.

Lifetime example

An address is not an eternal permission token. After `free(p)`, or after an automatic object leaves its lifetime, dereferencing a stale pointer is undefined even if:

- its numeric bits have not changed;
- the virtual page is still mapped;
- the bytes still look familiar;
- the allocator has not reused the region.

The compiler reasons from object lifetime, while the MMU translates virtual addresses. Those are different layers.

Sanitizers are observers, not semantics

AddressSanitizer and UndefinedBehaviorSanitizer instrument selected operations. They catch many executed bugs, not every possible UB. A clean run does not prove a program is defined; an optimizer may also remove an operation before instrumentation can observe it. Use sanitizers alongside contracts, warnings, tests, and review.

---

4. C atomics: indivisibility plus ordering

Include `<stdatomic.h>` and use an atomic type:

```
atomic_uint jobs_done;
atomic_init(&jobs_done, 0U);
atomic_fetch_add_explicit(&jobs_done, 1U, memory_order_relaxed);
```

Atomic operations prevent torn/conflicting access to that atomic object and participate in the language's ordering model. Atomic does not necessarily mean one machine instruction or lock-free execution. Ask `atomic_is_lock_free` when that property matters; the implementation may use library support or locks.

Data race

In simplified form, two conflicting actions in different threads constitute a data race when:

- they access the same memory location;
- at least one modifies it;
- they are not atomic;
- neither happens before the other.

A data race is undefined behavior. Cache coherence may eventually propagate bytes, but coherence is not a substitute for the C happens-before relation.

`memory_order_relaxed`

Relaxed operations remain atomic and each atomic object has a modification order, but relaxed ordering does not publish surrounding ordinary data.

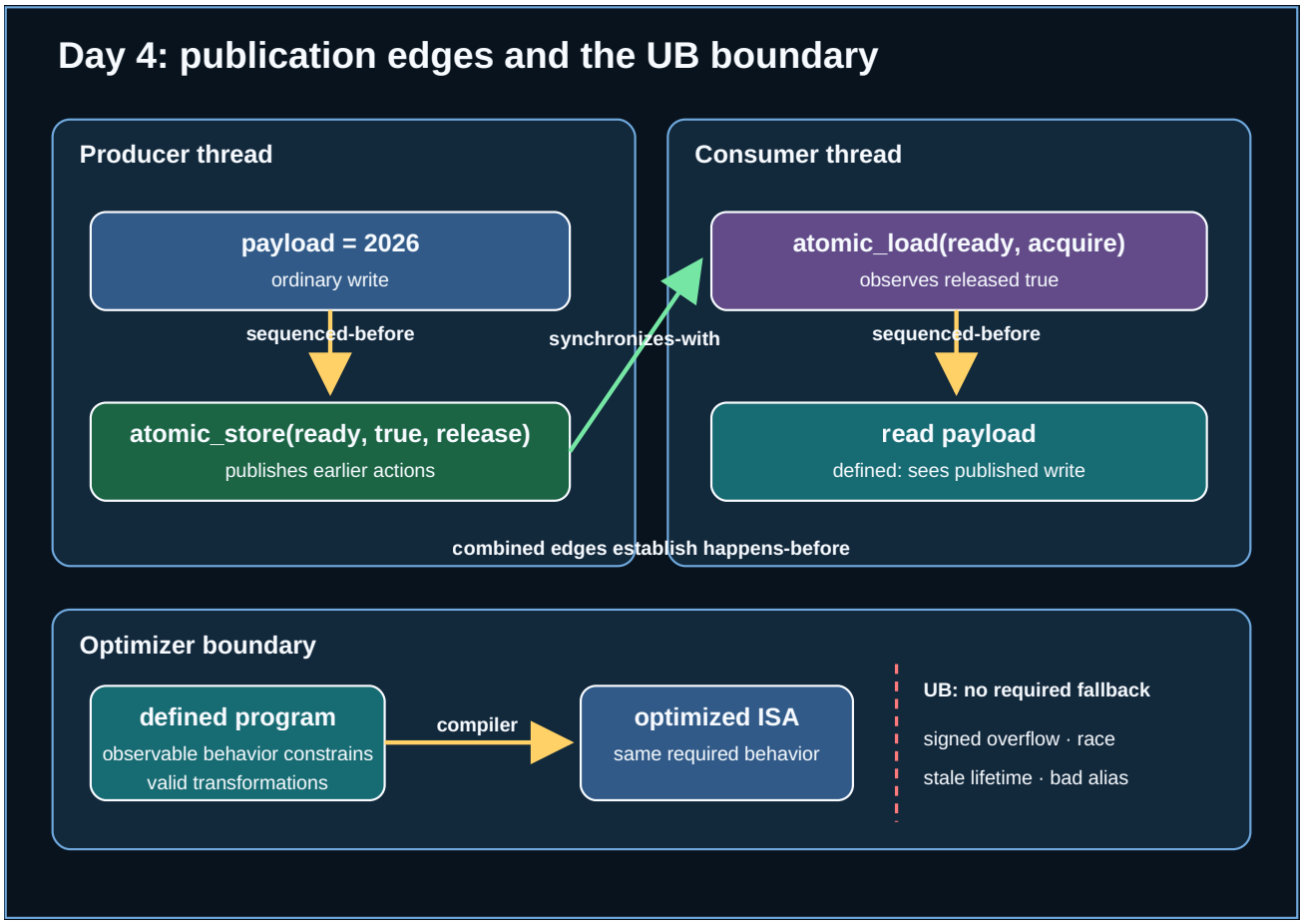
Use it when only the atomic value itself matters, such as an independent statistics counter:

```
atomic_fetch_add_explicit(&requests, 1U, memory_order_relaxed);
```

Release/acquire publication

```
producer                                  consumer
ordinary write: payload = 2026
release store: ready = true     ──────►   acquire load observes true
                                            ordinary read: payload

producer actions sequenced before release
      happen before
consumer actions sequenced after matching acquire
```



Release/acquire creates a publication edge; undefined behavior removes the source guarantees an optimizer preserves.

When an acquire operation reads the value from the relevant release sequence, the release synchronizes with the acquire. Earlier producer actions then happen before later consumer actions. This is why the non-atomic payload in the Day 4 lab can be read safely after the matching acquire.

`memory_order_seq_cst`

Sequentially consistent operations provide the strongest standard ordering: in addition to acquire/release effects as applicable, sequentially consistent operations participate in a single total order consistent with required thread order. It is a useful default while learning, but it does not turn a multi-step algorithm into a transaction.

Learn in this order:

1. make shared access race-free;
2. state the invariant and required happens-before edges;
3. begin with mutexes or sequential consistency where appropriate;
4. weaken ordering only with a proof and measurements.

Avoid `memory_order_consume` in introductory designs; its specification and implementation history make acquire the practical teaching baseline.

---

5. C model, compiler, CPU, and kernel

One source operation can map differently by target:

```
C acquire load
  → compiler ordering constraints
  → perhaps an ordinary load on a sufficiently strong ISA
  → perhaps a load-acquire instruction or barrier on another ISA
```

That difference does not make acquire “free” in the language model. It means the target ISA already supplies some required ordering.

Keep these statements separate:

- **C rule:** a data-race-free program uses language/library synchronization.
- **Compiler mapping:** preserves the C contract while optimizing.
- **ISA rule:** defines architectural load/store ordering and atomic instructions.
- **microarchitecture:** implements the ISA with caches, buffers, and coherence.
- **kernel/device rule:** supplies APIs for locks, MMIO, DMA, interrupts, and architecture barriers.

Linux kernel code uses a kernel memory model and primitives including locks, atomics, barriers, `READ_ONCE()`, and `WRITE_ONCE()`. Their meanings come from kernel documentation and supported compiler behavior, not directly from ISO C atomics. Conversely, copying kernel idioms into portable userspace without their contracts is unsafe.

`volatile` remains useful for its intended observable-access role. It is not a weaker spelling of `_Atomic`, and `_Atomic` is not a general device-register API.

Architecture lens — x86-64, AArch64, and RISC-V

The same C atomic order can compile differently. x86-64's relatively strong TSO-style ordering often lets acquire loads and release stores use ordinary load/store instructions, while AArch64 commonly uses acquire/release instructions or barriers. RISC-V uses RVWMO plus fences, `aq`/`rl` bits, AMOs, or LR/SC when the selected ISA extensions provide them.

Those mappings do not weaken or strengthen the source-level contract: a C data race remains undefined on x86-64, and release/acquire publication has the same C meaning on all three. Lock-free status and the instruction sequence are implementation and target properties, so check rather than infer them. See the architecture comparison.

---

6. Observe

Qualifiers and explicit UB modes

Build the defined default:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g \
  03-c-toolchain-and-startup/challenges/day-004-qualifiers-ub-lab.c \
  -o /tmp/week3-day4-qualifiers

/tmp/week3-day4-qualifiers
```

Compare generated code:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -S \
  03-c-toolchain-and-startup/challenges/day-004-qualifiers-ub-lab.c \
  -o /tmp/day4-O0.s
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -S \
  03-c-toolchain-and-startup/challenges/day-004-qualifiers-ub-lab.c \
  -o /tmp/day4-O2.s
diff -u /tmp/day4-O0.s /tmp/day4-O2.s
```

The unsafe modes are opt-in only. Rebuild with diagnostics before selecting one:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O1 -g \
  -fsanitize=address,undefined -fno-omit-frame-pointer \
  03-c-toolchain-and-startup/challenges/day-004-qualifiers-ub-lab.c \
  -o /tmp/week3-day4-ub

/tmp/week3-day4-ub overflow
/tmp/week3-day4-ub lifetime
```

`restrict-overlap` deliberately breaks a language contract, but a sanitizer is not required to diagnose that class. Never infer validity from one output.

Safe atomics

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g -pthread \
  03-c-toolchain-and-startup/challenges/day-004-atomics-lab.c \
  -o /tmp/week3-day4-atomics

/tmp/week3-day4-atomics
```

Predict first:

1. What exact counter value is required?
2. Why is relaxed enough for that counter?
3. Which operation publishes `payload`?
4. Would replacing `ready` with `volatile` preserve the C guarantee?
5. Must `atomic_uint` report lock-free?

Optional: run the safe atomic lab under ThreadSanitizer where the installed compiler/runtime supports it:

```
clang -std=c17 -Wall -Wextra -Wpedantic -O1 -g -pthread \
  -fsanitize=thread \
  03-c-toolchain-and-startup/challenges/day-004-atomics-lab.c \
  -o /tmp/week3-day4-tsan
/tmp/week3-day4-tsan
```

Runtime support varies by platform; tool availability is not part of the C guarantee.

---

7. Mastery — Explain · Draw · Observe · Build

Explain

- Explain `const`, `volatile`, and `restrict` without using “makes it safe.”
- Explain why volatile polling is not thread synchronization.
- Distinguish undefined, unspecified, and implementation-defined behavior.
- Explain why signed machine addition wrapping does not define signed C overflow.
- Explain why an atomic operation need not be lock-free.
- Explain the difference between atomicity and ordering.

Draw

Draw two diagrams:

1. C abstract machine → optimizer → ISA → microarchitecture → device/kernel API. Label which layer owns each guarantee.
2. Producer payload write → release store → acquire load → payload read. Mark **sequenced-before**, **synchronizes-with**, and **happens-before**.

Observe

Save:

- warning-clean GCC and Clang build commands for both labs;
- one `-O0`/`-O2` assembly comparison;
- one sanitizer report from an explicitly selected UB mode;
- atomic output showing the exact counter and publication value.

For each observation, state what it demonstrates and what it does **not** prove.

Build

Extend the atomic lab with a fixed-size one-producer/one-consumer mailbox:

- payload writes happen before consumption;
- no ordinary shared access races;
- shutdown is represented explicitly;
- every memory order has a one-sentence justification.

First implement it with a mutex and condition variable. Then implement the single-slot version with release/acquire atomics. Compare correctness arguments before performance.

Why notebook

1. Why can `const int *` still observe a changed value?
2. Why can volatile be correct for an implementation-defined device access but wrong for thread publication?
3. Why is a violated `restrict` promise the caller's correctness bug?
4. Why does UB allow optimization before any bad CPU instruction executes?
5. Why can an address remain mapped after its C object lifetime ends?

6. Why does relaxed ordering suit a standalone counter but not payload publication?
7. Why can acquire compile to different instructions on x86-64 and AArch64?
8. Why do kernel memory-ordering primitives require their own documentation?

References used selectively

Current language and compiler references:

- ISO/IEC 9899:2024 (C23), the current published C standard: https://www.iso.org/standard/82075.html
- WG14 N3220 working draft, especially 5.1.2.4 (multi-threaded executions and data races), 6.2.4 (object lifetime), 6.7.4 (type qualifiers), and 7.17 (atomics): https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf
- GCC 16.2 manuals, “When is a Volatile Object Accessed?”, restricted pointers, C implementation-defined behavior, optimizer options, and instrumentation: https://gcc.gnu.org/onlinedocs/gcc/Volatiles.html https://gcc.gnu.org/onlinedocs/gcc/Restricted-Pointers.html https://gcc.gnu.org/onlinedocs/gcc/C-Implementation.html https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html
- Clang documentation for the language extensions and sanitizers: https://clang.llvm.org/docs/LanguageExtensions.html https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html https://clang.llvm.org/docs/AddressSanitizer.html https://clang.llvm.org/docs/ThreadSanitizer.html

Current systems references:

- Linux kernel documentation, “Why the `volatile` type class should not be used” (with its documented exceptions): https://docs.kernel.org/process/volatile-considered-harmful.html
- Linux kernel memory-barrier documentation: https://docs.kernel.org/core-api/wrappers/memory-barriers.html
- Linux kernel MMIO access documentation: https://docs.kernel.org/driver-api/device-io.html
- Linux Kernel Memory Model explanation and tooling: https://docs.kernel.org/dev-tools/lkmm/index.html

Next bridge: today the compiler transformed one translation unit under C rules. Day 5 follows the artifacts through preprocessing, compilation, assembly, symbols, sections, and relocations.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Atomics Lab

Format: C lab

What you will do: Day 4 atomics lab: defined communication between POSIX threads.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-004-atomics-lab.c -lm -o /tmp/day-004-atomics-lab && /tmp/day-004-atomics-lab
```

- View lab source on GitHub

Put it into practice — Qualifiers Ub Lab

Format: C lab

What you will do: Day 4 qualifiers and undefined-behavior lab.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-004-qualifiers-ub-lab.c -lm -o /tmp/day-004-qualifiers-ub-lab && /tmp/day-004-qualifiers-ub-lab safe
```

- View lab source on GitHub

Day 5 — From Source to Object File

Curriculum alignment: Stage 6 — Compilation, Linking, ELF, and Program Startup · Tracker Week 19

Target time: approximately 2½–3 hours

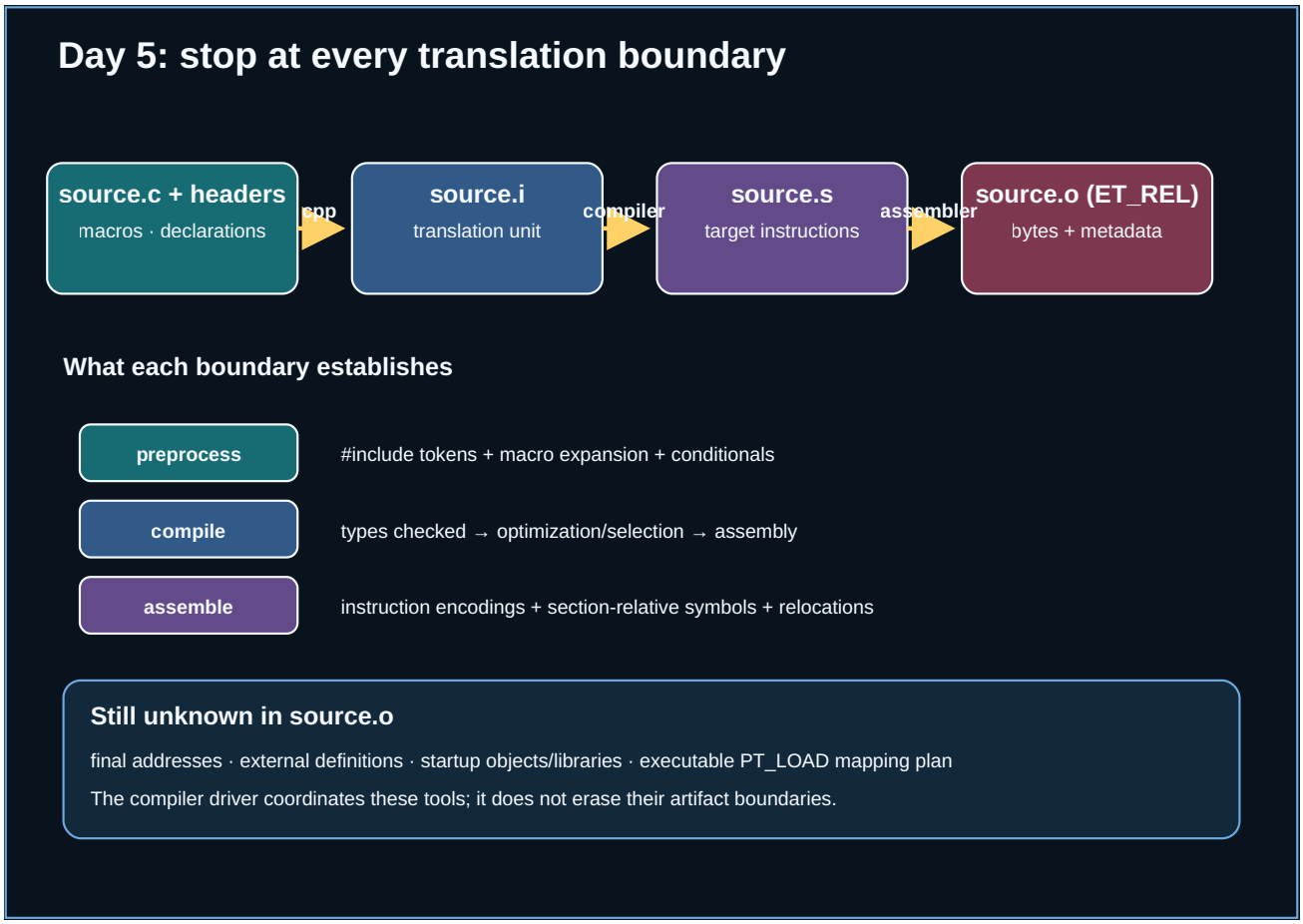
- Why translation is staged: 25 minutes
- Preprocessor, front end, optimizer, and back end: 55 minutes
- Assembly, ELF sections, symbols, and relocations: 45 minutes
- Pipeline lab and mastery evidence: 55 minutes

> You wrote `printf("%d\n", transform(5));`. Before a linker has produced a program, what concrete information can already exist—and what must still be unknown?

That question repairs a misleading picture: “the compiler turns C into an executable.” In ordinary Linux work, several tools transform different representations under different contracts. The compiler driver coordinates them, which is convenient, but convenience should not erase the boundaries.

Today we stop after each boundary:

```
source + headers
    ↓ preprocessing
translation unit
    ↓ parsing, semantic analysis, optimization, code generation
assembly text
    ↓ assembly
relocatable ELF object
```



Each stopping point is a different artifact produced under a different tool contract.

The object already contains machine instructions and data. It is not yet a complete process image: some addresses and external definitions remain for a linker to resolve.

---

1. Why engineers separated the stages

Early programmers entered numeric instructions or assembly mnemonics. Higher level languages made one source program portable across machines, but that created two distinct problems:

1. understand the language and choose machine operations;
2. encode those operations using one target ISA and object format.

Assemblers already solved the second problem. Early compilers could therefore emit assembly and reuse the assembler. Separate compilation solved another practical constraint: a large program should not need complete recompilation when one source file changes. Each source file becomes an object; a later tool combines objects.

Unix reinforced this compositional design. Small tools and stable textual or binary interfaces made each representation observable. Modern GCC and Clang perform far more internally than the old pipeline, but `-E`, `-S`, and `-c` still expose useful stopping points.

The compiler driver is a coordinator

Running:

```
gcc hello.c -o hello
```

does not imply one indivisible operation. The driver chooses preprocessors, compiler components, assembler, linker, startup objects, default libraries, search directories, target options, and output names. Ask it to show the commands without running them:

```
gcc -### hello.c -o hello
clang -### hello.c -o hello
```

Exact commands vary by distribution and compiler build. That variation is evidence that the driver is policy around several contracts.

---

2. Translation units begin with preprocessing

A C source file is not, by itself, the complete input parsed as C. After comments are replaced and preprocessing directives are handled, the compiler sees a **preprocessing translation unit**.

```
#include <stdio.h>
#define SCALE(v) ((v) * 3)
printf("%d\n", SCALE(input));
```

`#include` performs token-level inclusion of a header. `#define` establishes a macro substitution rule. Conditional directives choose tokens:

```
#if defined(DEBUG)
    debug_log(value);
#endif
```

Observe the result:

```
gcc -E -dD source.c > source.i
gcc -E -P source.c > source-without-line-markers.i
```

`-E` stops after preprocessing. `-P` makes an easier reading copy but removes line markers useful to later diagnostics. A header normally supplies declarations, types, macros, and inline definitions—not a magical reference to another compiled file.

Why macro bugs feel different

The compiler proper checks expanded tokens, not the author's intended macro abstraction:

```
#define SQUARE(x) x * x
SQUARE(a + b)             /* expands to a + b * a + b */
```

Parentheses repair this case, but evaluating an argument more than once remains dangerous. Use the preprocessed output when a diagnostic appears to refer to code you did not seem to write.

Translation unit is not object file

A translation unit is the language-level result of preprocessing one primary source file and its included content. An object file is a target-specific ELF container produced later. One often leads to one, but they are not synonyms.

---

3. Inside compilation: tokens to machine choices

Compiler implementations differ, and many optimization passes blur a simple linear story. This model is still useful:

```
characters
  → lexer: tokens
  → parser: syntax tree / AST
  → semantic analysis: types, scopes, constraints
  → intermediate representation (IR)
  → optimization and lowering
  → target instruction selection and register allocation
  → assembly or machine-code emission
```

Lexer and parser

The lexer recognizes tokens such as identifiers, keywords, literals, and operators. The parser checks grammatical structure. In `a + b * c`, the tree records multiplication beneath addition, preserving precedence.

An **abstract syntax tree** describes source-language structure without every punctuation token. Semantic analysis then asks questions grammar cannot:

- was this identifier declared in this scope?
- may this operand be dereferenced?
- what conversions apply?
- is this call compatible with its declaration?

Syntax can be valid while constraints fail:

```
int *pointer;
pointer = 42;            /* parseable, but diagnostically incompatible */
```

Why an IR exists

An intermediate representation gives many source languages and many target architectures a shared middle ground. It makes control flow, data flow, and operations explicit enough for analysis and transformation.

Do not equate LLVM IR, GCC's GIMPLE, an AST, assembly, and machine code. They answer different questions. IR is an implementation representation, not the C abstract machine and not an ISA contract.

Clang can expose one form:

```
clang -S -emit-llvm -O0 source.c -o source.ll
clang -Xclang -ast-dump -fsyntax-only source.c
```

These commands are compiler-specific observation windows, not standardized C stages.

Optimization and code generation

The optimizer preserves the behavior the language requires, not the visual shape of source. Under C's rules it may fold constants, remove unreachable or dead work, inline calls, and rearrange operations. Undefined behavior can remove constraints the programmer assumed existed.

Compare:

```
gcc -S -O0 source.c -o source-O0.s
gcc -S -O2 source.c -o source-O2.s
diff -u source-O0.s source-O2.s
```

The back end chooses target instructions, registers, addressing modes, and instruction scheduling. ABI rules constrain calling convention and object layout; the ISA constrains encodings; microarchitecture can influence cost choices.

---

4. The assembler produces relocatable ELF

Assembly text names instructions, labels, directives, and sections. The assembler turns it into a **relocatable object file**, commonly ELF type `ET_REL` on Linux:

```
gcc -c source.s -o source.o
file source.o
readelf -h source.o
```

The `.o` file is binary, but it is not merely “machine code.” It is a structured handoff package.

Sections group material by purpose

Typical sections include:

- `.text`: executable instructions;
- `.rodata`: read-only constants;
- `.data`: initialized writable objects;
- `.bss`: zero-initialized storage described mostly by size, not stored bytes;
- `.symtab` and `.strtab`: full linking symbol metadata and names;
- `.rela.text` or similar: relocations applying to instructions;
- `.debug_*`: optional debugger information;
- `.note.GNU-stack`: toolchain metadata concerning stack policy.

Inspect them:

```
readelf -SW source.o
objdump -s -j .rodata source.o
size -A source.o
```

Section placement can vary with options. With `-ffunction-sections`, functions may receive separate text sections. Names are conventions interpreted by the toolchain, not physical memory compartments already mapped into a process.

Symbols name definitions and unresolved references

A symbol-table entry can describe a function or object, its binding, section, size, and value relative to that section. A defined local `static` function and an externally visible function have different binding. An external call may remain undefined:

```
nm -C source.o
readelf -sW source.o
```

Common `nm` letters include `T/t` for text, `D/d` for initialized data, `B/b` for zero-initialized data, `R/r` for read-only data, and `U` for an undefined reference. Case often distinguishes global from local binding. Treat these as display conventions; confirm details with `readelf`.

Relocations record unfinished address calculations

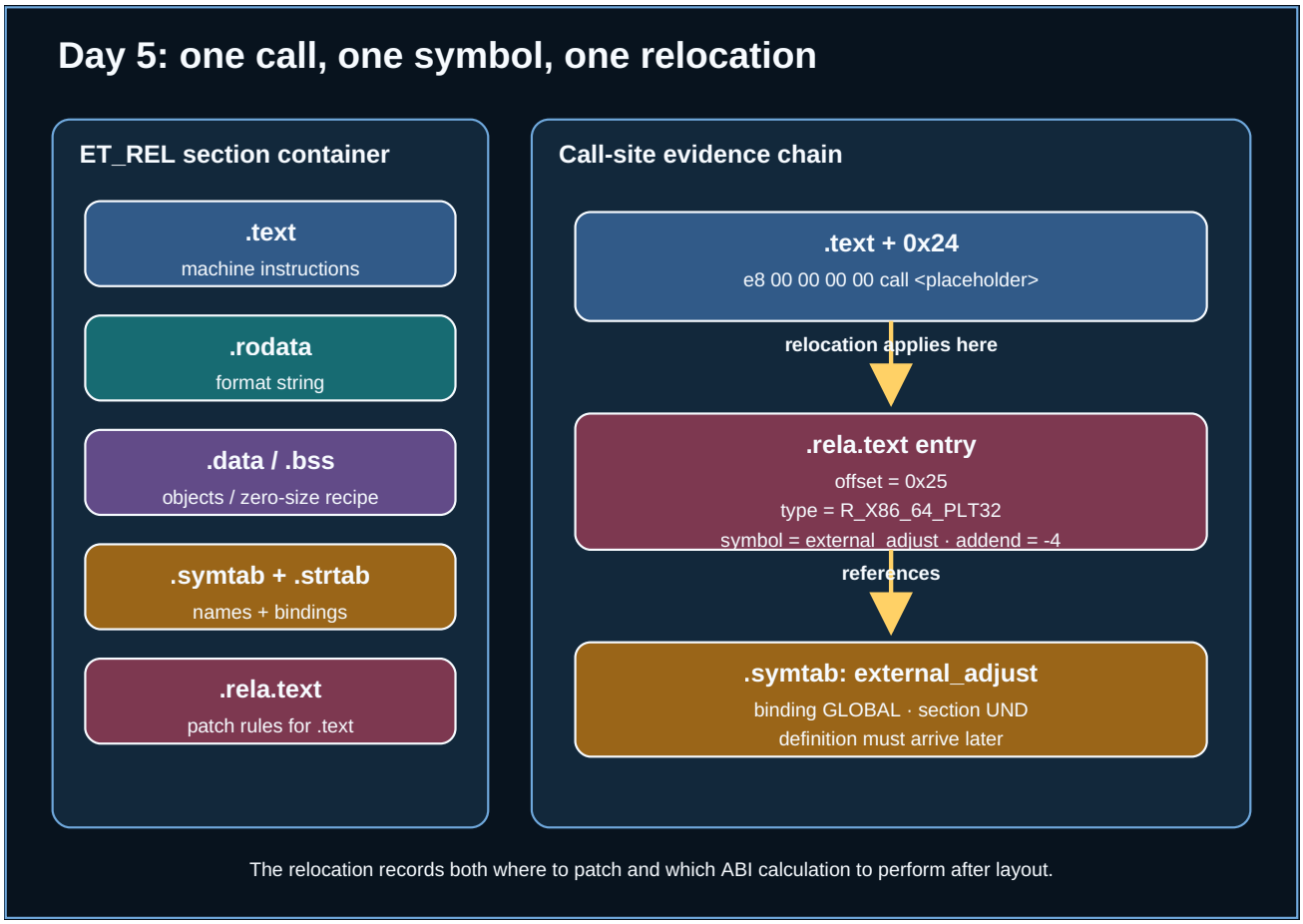
Suppose an instruction calls `external_adjust`, but that function will arrive from another object or library. The assembler cannot know the final distance. It emits:

1. an instruction encoding with a placeholder/addend;
2. a symbol reference;
3. a relocation record saying how a linker should compute and place the final field.

On x86-64, a direct relative call commonly uses a relocation such as `R_X86_64_PLT32`; accesses may use PC-relative or GOT-related relocation types. The exact type depends on code model, visibility, PIC options, and toolchain. Observe rather than memorize one result:

```
readelf -Wr source.o
objdump -dr source.o
```

`objdump -dr` is powerful because it places relocation annotations beside the instruction bytes they qualify.



A relocation ties one instruction field to a symbol and an ABI-defined calculation.

Architecture lens — x86-64, AArch64, and RISC-V

ELF is a shared container format, not a shared instruction or relocation language. `e_machine` identifies targets such as `EM_X86_64`, `EM_AARCH64`, or `EM_RISCV`, and each target ABI defines its own relocation types, instruction fields, range checks, and addend rules. A relocation that repairs an x86-64 relative call is therefore not portable metadata for an AArch64 or RISC-V call sequence.

AArch64 and RISC-V address construction also commonly spans paired instructions and relocations, while linkers may relax eligible sequences under target-specific rules. The C translation unit can remain portable even though its `.o` is unavoidably target-specific; inspect `readelf -h` and `readelf -r` together. See the architecture comparison.

---

5. Lab: stop at every stage

From the repository root:

```
python3 03-c-toolchain-and-startup/challenges/day-005-object-lab.py
```

The lab creates a private temporary directory, runs the available `gcc` or `clang`, and removes generated artifacts unless `--keep` is requested. The source intentionally refers to `printf`, `external_adjust`, and `shared_counter` without defining all of them. Compilation must succeed; linking that object alone must fail.

Before running, predict:

1. which macro text disappears into expanded C;
2. which strings and initialized objects occupy data sections;
3. which names will show as defined, local, global, or undefined;
4. which instructions need relocation records;
5. why `.bss` storage need not consume the same number of file bytes.

Then rerun manually:

```
gcc -E 03-c-toolchain-and-startup/challenges/day-005-object-demo.c |
  less
gcc -S -O0 03-c-toolchain-and-startup/challenges/day-005-object-demo.c -o /tmp/demo.s
gcc -c -O0 03-c-toolchain-and-startup/challenges/day-005-object-demo.c -o /tmp/demo.o
readelf -SWsWr /tmp/demo.o
objdump -dr /tmp/demo.o
```

Delete the `/tmp` artifacts when finished.

Build the missing provider

Write `/tmp/provider.c` defining:

```
int shared_counter = 4;
int external_adjust(int value) { return value - 1; }
```

Compile it separately, link both objects, and predict the output before running. This is a manual multi-file project: two translation units, two objects, one final executable.

---

6. Linux and kernel connections

Linux user programs, shared libraries, the kernel, and loadable modules all depend on object metadata, but they do not all use identical link/runtime rules.

- Kernel compilation creates many `.o` files and archives before the final `vmlinux` link.
- A loadable module is an ELF relocatable object with module-specific metadata and relocations that the kernel module loader processes.
- BPF toolchains can emit ELF objects whose sections and relocation metadata are consumed by loaders with domain-specific meanings.
- Debuggers map addresses back to source using optional DWARF sections.

Do not infer that the kernel invokes the normal userspace dynamic linker for modules. The shared object format enables reuse; the consumer and policy are different.

---

7. Mastery: Explain · Draw · Observe · Build

The tracker checkpoint is exact:

> **Explain what information exists in a `.o` file before linking.**

A complete answer names machine instructions and data bytes, section organization, symbol definitions and unresolved references, relocation records and addends, attributes/notes, and optional debug metadata. It also states what is missing: final virtual addresses, resolution of every external symbol, startup objects/libraries, and the final loadable segment plan.

Explain

Without notes:

- describe preprocessing, compilation, and assembly;
- distinguish source file, translation unit, assembly, and object file;
- explain lexer, parser/AST, semantic analysis, IR, optimization, and code generation without claiming every compiler uses one fixed sequence;
- explain why undefined symbols can be valid in `.o` and errors at final link.

Draw

Draw the pipeline with the input, output, responsible tool, and one preserved piece of information at every arrow. Beside it, draw an object container with `.text`, `.rodata`, `.data`, `.bss`, symbols, and relocations. Connect one call instruction to its symbol and relocation.

Observe

Save:

- one preprocessed macro expansion;
- one source expression and its `-O0` / `-O2` assembly;
- one symbol-table entry;
- one relocation beside the instruction it repairs.

Build

Complete the provider object, link the two-file program, then change `external_adjust` to another translation unit. Rebuild only the changed object and relink. Explain why the unchanged source did not need recompilation.

Why? notebook

1. Why does `#include` not mean “link this library”?
2. Why can syntactically valid C still fail semantic analysis?
3. Why is IR neither an ISA nor the C abstract machine?
4. Why does `.bss` occupy memory without storing equivalent zero bytes?
5. Why can an object contain real instructions but still not be runnable?
6. Why must a relocation identify both a location and a calculation rule?
7. Why are section names useful before any process mapping exists?

References used selectively

Local:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 7, §§7.1–7.7, PDF pages 705–725, for compiler-driver stages, relocatable objects, symbols, resolution, and relocation: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`

Current primary references:

- GCC 16.2 manuals, “Options Controlling the Kind of Output” and “Overall Options” (`-E`, `-S`, `-c`, `-v`, `-###`): https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html
- Clang documentation, command-line reference and AST introduction: https://clang.llvm.org/docs/ClangCommandLineReference.html and https://clang.llvm.org/docs/IntroductionToTheClangAST.html
- GNU Binutils manuals for `as`, `readelf`, `nm`, and `objdump`: https://sourceware.org/binutils/docs/
- System V ABI generic ELF specification, object files, sections, symbols, and relocation: https://gabi.xinuos.com/

Next bridge: the object deliberately leaves questions open. Day 6 asks how a linker resolves those names, performs those calculations, and chooses between copying library code now and binding shared code later.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Object Demo

Format: C lab

What you will do: Use this object demo exercise to test the section's model.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -c 03-c-toolchain-and-startup/challenges/day-005-object-demo.c
-o /tmp/day-005-object-demo.o && readelf -h -S /tmp/day-005-object-demo.o
```

- View lab source on GitHub

Put it into practice — Object Lab

Format: Python lab

What you will do: Build and inspect each translation stage without modifying the source tree.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 03-c-toolchain-and-startup/challenges/day-005-object-lab.py
```

- View lab source on GitHub

Day 6 — Linking and Libraries

Curriculum alignment: Stage 6 — Compilation, Linking, ELF, and Program Startup · Tracker Week 20

Target time: approximately 2½–3 hours

- Why separate objects need a linker: 25 minutes
- Resolution, archives, and relocation: 50 minutes
- Shared objects, PIC, GOT, and PLT: 50 minutes
- Search/versioning risks, lab, and mastery: 50 minutes

> Yesterday, `main.o` contained a call instruction, an undefined symbol named `external_adjust`, and a relocation. Today, who decides which bytes that name denotes—and when?

Linking is the act of composing separately produced objects into a new object or executable while preserving the meaning of names and addresses. Libraries change where definitions come from and when binding happens, but they do not abolish symbol resolution or relocation.

```
objects + archives + shared-object requirements + linker policy
    ↓ resolve names
    ↓ choose and lay out input sections
    ↓ apply static relocations / preserve dynamic relocations
executable or shared object
```

---

1. The engineering need: compile separately, compose later

Separate compilation makes change affordable. A project with a thousand source files can rebuild one changed translation unit and relink. Libraries make reused implementations distributable without pasting source into every program.

That design creates obligations:

- two objects may define the same external name;
- one object may require a name no input provides;
- machine instructions may contain address-dependent fields;
- an archive should contribute only members actually needed;
- a shared-library definition may not have a final runtime address yet.

The linker turns those obligations into either a coherent result or a useful failure.

The linker does not understand your whole C program

A traditional ELF linker primarily sees sections, symbols, relocations, and options—not C scopes and expressions. Link-time optimization can preserve a compiler IR across translation units, but that is an additional compiler- linker protocol. Start with the ordinary object model.

---

2. Symbol resolution: make each reference choose a definition

At a high level, resolution associates symbol references with definitions. ELF symbol binding includes local, global, and weak forms.

- **Local** symbols are confined to one object for linking purposes.
- A **global/strong** definition can satisfy external references.
- A **weak** definition can serve as a fallback and is generally overridden by a strong definition.

The practical rules are nuanced by symbol visibility, common symbols, linker options, language conventions, and dynamic interposition. Avoid the folk rule “global variables are always strong.” Modern GCC defaults to `-fno-common`; multiple tentative definitions that old toolchains merged may now produce a duplicate-definition error.

Diagnose the inputs:

```
nm -A *.o
readelf -sW one.o
```

Typical failures:

```
undefined reference to `compute'
multiple definition of `configuration'
```

The first means no selected input supplied a usable definition. The second means incompatible selected inputs supplied more than one.

Name identity includes ABI assumptions

Matching a symbol name is not enough for correctness. The producer and consumer must agree on calling convention, parameter/return representation, object size/alignment, and data layout. C headers carry declarations so both translation units compile against one contract. ELF resolution cannot detect every mismatched prototype.

---

3. Archives are indexed bags of object files

A static archive is commonly created with:

```
ar rcs libmathops.a add.o multiply.o
```

It is not one monolithic compiled function. It is a container of members plus an index. While scanning the link command, the linker extracts archive members needed to satisfy unresolved symbols. This explains why order can matter with traditional one-pass archive processing:

```
gcc main.o -L. -lmathops       # conventional useful order
gcc -lmathops main.o           # may leave main's needs unseen when scanned
```

Circular archive dependencies may require `--start-group/--end-group`, but those rescans cost time and can hide poor dependency structure. Prefer clear acyclic library interfaces where practical.

What “static linking” means here

With an archive, selected object members are copied into the link result and their relocations are processed. This does **not** imply that every byte of every archive member or every system facility becomes self-contained.

`-static` requests a much broader fully static link using static forms of default libraries. It can fail when static libraries are absent, enlarge the binary, and interact poorly with facilities designed around NSS, plugins, locale data, or dynamic loading. The lesson lab links a private archive but does not require a fully static glibc executable.

---

4. Relocation: calculate addresses after layout

After selecting input sections, the linker assigns output locations. A relocation tells it:

```
place P: where to patch
symbol S: which definition is involved
addend A: constant adjustment
type: formula and encoded-field constraints
```

A PC-relative relocation may conceptually compute `S + A - P`; an absolute one may compute `S + A`. The actual ABI defines each formula, field width, overflow check, and semantics.

```
readelf -Wr main.o
ld --verbose                # inspect default script; output is long
readelf -SW program
objdump -dr program
```

Relocation is not “moving the function.” It is repairing address-dependent fields after selection and layout. Some relocations are completed by the static linker. Others are emitted into the dynamic executable for the runtime loader.

---

5. Shared objects postpone part of the binding

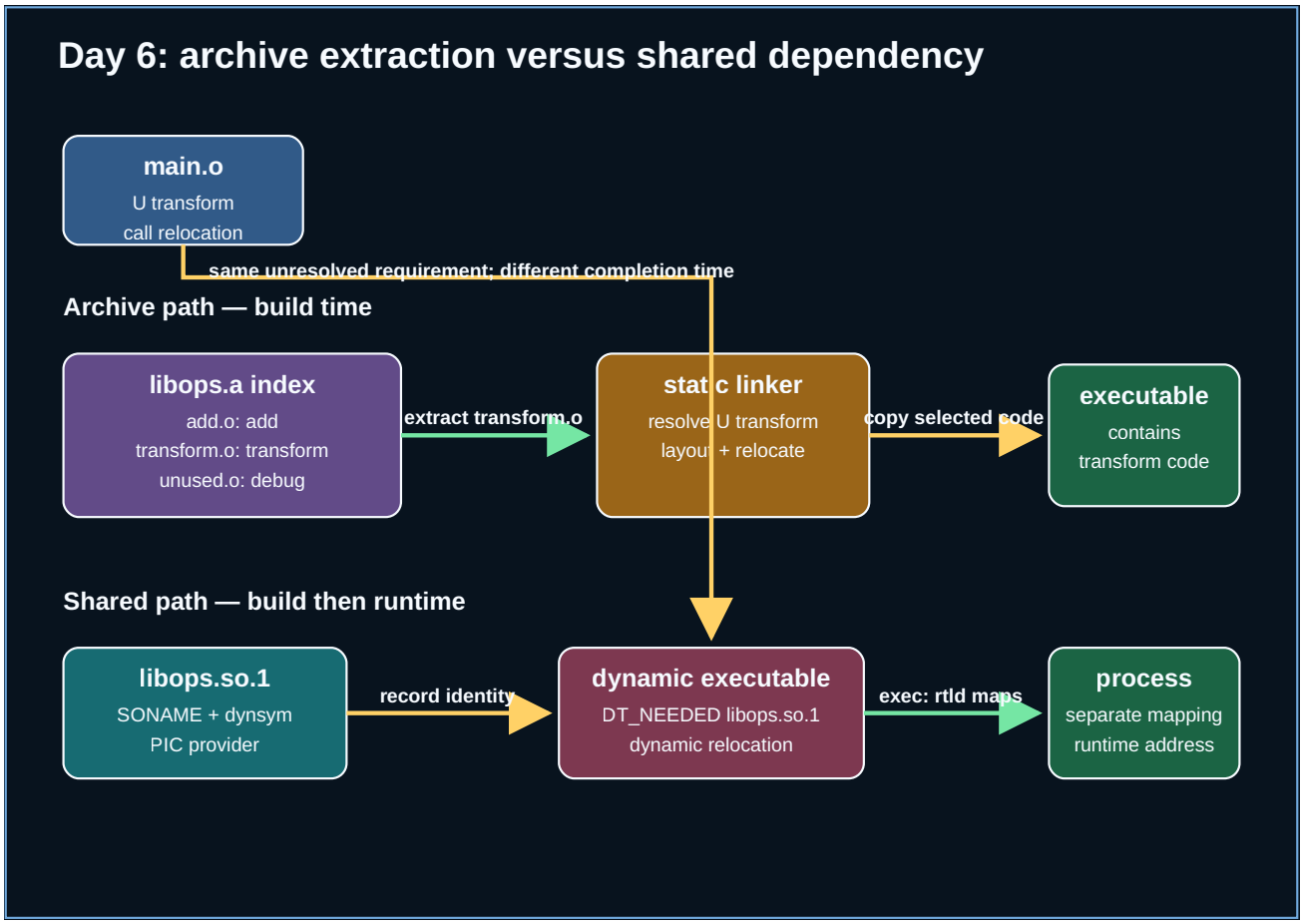
Build position-independent code and a shared object:

```
gcc -fPIC -c mathops.c -o mathops.pic.o
gcc -shared -Wl,-soname,libmathops.so.1 \
    -o libmathops.so.1.0 mathops.pic.o
```

An executable dynamically linked against it normally records a dependency such as `DT_NEEDED: libmathops.so.1`, not a copy of all library code.

At execution time, the ELF interpreter/dynamic linker maps the required objects, resolves dynamic symbols under its lookup rules, and applies remaining relocations. Sharing read-only code pages across processes saves memory and lets compatible library updates serve multiple programs.

The costs include startup work, a larger compatibility surface, runtime search policy, and the possibility of loading an unintended library.



Archive members are selected and copied at link time; shared definitions are mapped and completed at runtime.

PIC: code that tolerates different load addresses

Position-independent code avoids embedding unnecessary fixed absolute addresses in executable instructions. On x86-64 it can use RIP-relative addressing for nearby data. External/preemptible definitions need additional indirection because another ELF object may supply the chosen definition.

PIC is not “code with no addresses.” It is code whose address calculations remain valid when the object is mapped at different bases, with loader-fixed tables where needed.

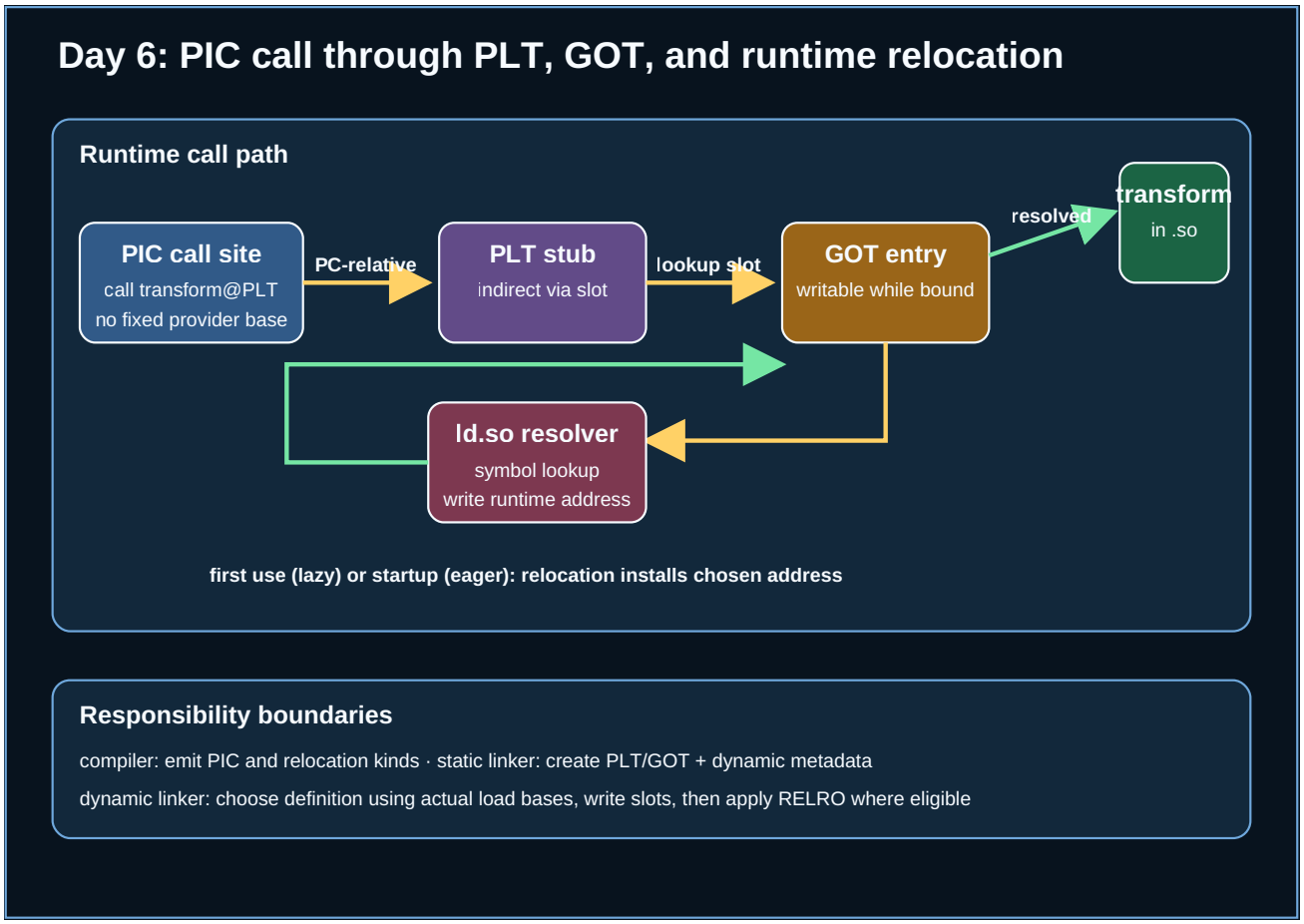
GOT and PLT at a high level

The **Global Offset Table (GOT)** contains address slots used by position-independent references. The dynamic linker writes resolved runtime addresses into relevant entries.

The **Procedure Linkage Table (PLT)** provides call stubs for external functions. Depending on toolchain options and binding policy, a call can pass through a PLT entry whose GOT state directs it to the dynamic resolver on first use and to the resolved function later.

```
call site → PLT entry → GOT slot → function
                          │
                          └→ resolver before binding
```

Not every external call must use a classic PLT. `-fno-plt`, symbol visibility, direct binding, architecture, and linker optimizations alter the shape. Modern systems may use eager binding (`-z now` or `LD_BIND_NOW=1`). Observe your binary:



PIC keeps the call site movable while runtime relocation installs the selected function address.

```
objdump -d -j .plt program
readelf -Wr program
readelf -dW program
```

Architecture lens — x86-64, AArch64, and RISC-V

PIC, GOT, and PLT are ELF ABI concepts with target-specific realizations. x86-64 commonly uses RIP-relative references; AArch64 commonly builds PC-relative addresses with instruction pairs; RISC-V commonly uses `AUIPC`-based pairs. Their relocation names, PLT entry code, call ranges, and linker-generated veneers or stubs differ.

The linker may also relax a GOT or call sequence when visibility, range, and interposition rules permit, so one disassembly is not a universal PLT template. Keep the portable C declaration and ABI compatibility question separate from the mechanism chosen by a particular target linker. See the architecture comparison.

Security connection: RELRO

GOT entries must be writable while relocations are applied. RELRO lets the loader make relocation-related regions read-only afterward. Partial and full RELRO differ; `-Wl,-z,relro,-z,now` enables eager binding so more GOT state can be protected. This is hardening, not a substitute for memory safety.

---

6. Search paths, versions, and RPATH risks

Three names are easily collapsed:

- **link name:** `libmathops.so`, used by `-lmathops` at build time;
- **SONAME:** `libmathops.so.1`, the runtime compatibility identity;
- **real file:** `libmathops.so.1.0`, one installed implementation.

The SONAME major component should change when ABI compatibility is broken. Package managers usually maintain the symlink arrangement. Versioned symbols can support finer compatibility, but they do not repair arbitrary ABI changes.

Inspect requirements and resolution:

```
readelf -dW program | grep -E 'NEEDED|RPATH|RUNPATH'
ldd program
ld.so --list program        # where supported
```

Use `ldd` only on trusted binaries. Some historical/platform implementations may execute the target or its interpreter; `readelf -d` is the safer first inspection for unknown files.

RUNPATH/RPATH are security policy

Embedding `$ORIGIN` can make a relocatable private application bundle:

```
-Wl,-rpath,'$ORIGIN/../lib'
```

But a writable library directory lets an attacker replace code that the program will load. Empty path components and relative/current-directory searches are especially risky. Set-user-ID/set-group-ID execution invokes secure-execution restrictions, but do not treat those restrictions as a reason to ship unsafe search paths.

Prefer system package paths, a tightly controlled application directory, or explicit deployment policy. Never recommend global `LD_LIBRARY_PATH` as a permanent fix. It changes resolution for every launched dynamic program in that environment and is ignored/restricted in secure execution.

`DT_RPATH` and `DT_RUNPATH` also differ in precedence and transitivity. Modern GNU linkers generally emit RUNPATH for new dtags. Check the loader manual for the exact search order rather than memorizing a simplified list.

---

7. Lab: build static and shared forms

Run:

```
bash 03-c-toolchain-and-startup/challenges/day-006-linking-lab.sh
```

The script creates a temporary multi-file project, then:

1. compiles client and provider separately;
2. displays the client's unresolved symbols and relocations;
3. creates `libmathops.a` and links an archive-backed executable;
4. creates PIC, `libmathops.so.1.0`, and SONAME symlinks;
5. links a shared-object executable with experiment-only `$ORIGIN` RUNPATH;
6. inspects dependencies, PLT, and dynamic relocations;
7. intentionally triggers an undefined-symbol diagnostic;
8. removes the temporary directory.

Before running, predict which executable records `DT_NEEDED` for the private library and which contains copied archive member code.

Trigger duplicate definitions

In a temporary directory, add two files:

```
/* a.c */ int collision(void) { return 1; }
/* b.c */ int collision(void) { return 2; }
```

Compile both and link them with a caller. Record the diagnostic. Then repair the design—not by hiding the error with a linker flag, but by making one definition private with `static`, deleting the duplicate, or assigning distinct responsibilities.

Compare size honestly

Use `size`, `ls -l`, and `readelf -d`, but state what each measures. A small dynamically linked executable depends on bytes stored elsewhere. File size is not total runtime resident memory; shared pages may be counted in multiple process mappings while occupying physical memory once.

---

8. Linux and kernel connections

- Linux distribution ABI stability depends on SONAME and symbol/version discipline.
- Kernel `EXPORT_SYMBOL*` creates a controlled namespace for modules, but module symbol resolution is performed by the kernel loader, not glibc's dynamic linker.
- Kernel builds use archives and linker scripts heavily to order sections and collect init tables.
- Static PIE, PIE, shared objects, and kernel modules are all ELF uses with different relocation and runtime policies.

The common mechanism is metadata-directed composition. The authority that loads, resolves, and permits symbols differs.

---

9. Mastery: Explain · Draw · Observe · Build

The tracker checkpoint:

> **Trace how an external function call becomes bound to executable code.**

Give two traces.

Archive trace: source declaration → undefined object symbol + relocation → archive index selects provider member → static linker resolves symbol, lays out code, applies relocation → call reaches copied provider code.

Shared trace: source declaration → dynamic symbol/call mechanism → static link records `DT_NEEDED` and dynamic relocation/PLT-GOT state → loader maps the SONAME-selected object → dynamic lookup chooses a definition → relocation or lazy-binding update installs the runtime address → call reaches mapped shared code.

Explain

- explain symbol resolution and relocation;
- distinguish archive-backed, fully static, and dynamic linking;
- distinguish link name, SONAME, and real filename;
- explain PIC, GOT, and PLT without declaring one observed encoding universal;
- diagnose one undefined and one duplicate-symbol error.

Draw

Draw one executable assembled from `main.o` and one selected archive member. Then draw one process with an executable, loader, libc, and private shared object mapped at independent bases. Mark static versus runtime relocations.

Observe

Capture:

- `nm` before and after archive resolution;
- `DT_NEEDED`, SONAME, and RUNPATH;
- one GOT/PLT-related relocation;
- `ldd` or `ld.so --list` output for the trusted lab binary.

Build

Add `triple()` to the lab library. Rebuild both library forms. Change the implementation without changing its ABI and show that the shared executable uses the replacement without relinking. Then deliberately change the prototype and explain why a matching symbol name does not guarantee safety.

Why? notebook

1. Why can archive order affect resolution?
2. Why is relocation more precise than “fixing addresses”?
3. Why does PIC improve shareability?
4. Why can lazy binding need writable state?
5. Why is a SONAME an ABI promise rather than cosmetic filename decoration?
6. Why is `$ORIGIN` useful and dangerous?

7. Why is a dynamically linked binary's small file size an incomplete comparison?

References used selectively

Local:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 7, §§7.2–7.14, PDF pages 708–754, for static linking, resolution, relocation, archives, shared libraries, PIC, library interposition, and loading: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`

Current primary references:

- GNU `ld` manual, linker scripts, archive search, shared options, rpath, and `-z` hardening controls: https://sourceware.org/binutils/docs/ld/
- System V ABI generic ELF specification, symbol tables, relocation, dynamic linking, GOT, and PLT: https://gabi.xinuos.com/
- Linux man-pages `ld.so(8)`, including dependency search order, `$ORIGIN`, secure-execution mode, and dynamic-linker diagnostics: https://man7.org/linux/man-pages/man8/ld.so.8.html
- GCC options for code generation (`-fpic`, `-fPIC`, `-fpie`, `-fPIE`): https://gcc.gnu.org/onlinedocs/gcc/Code-Gen-Options.html

Next bridge: the linker can produce an executable and name its required interpreter. Day 7 follows `execve` from that file through kernel mappings, the dynamic linker, libc startup, constructors, `main`, and process exit.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Linking Lab

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 03-c-toolchain-and-startup/challenges/day-006-linking-lab.sh
```

- View lab source on GitHub

Day 7 — ELF Loading and `main`

Curriculum alignment: Stage 6 — Compilation, Linking, ELF, and Program Startup · Tracker Week 21

Target time: approximately 2½–3 hours

- ELF sections, segments, and the `execve` boundary: 45 minutes
- Kernel mappings, interpreter handoff, and relocations: 50 minutes
- Initial stack, libc startup, constructors, `main`, and exit: 45 minutes
- Read-only inspection lab and mastery trace: 40 minutes

> The shell has called `execve("./program", argv, envp)`, and the call succeeds. Which instruction runs next—and who arranged for it to run?

The answer is usually not `main`, and for a dynamically linked program it is not initially the executable's `_start` either. Linux replaces the calling process image, maps what the ELF program headers request, constructs an initial user stack, and enters the ELF interpreter named by `PT_INTERP`. That interpreter is the user-space dynamic linker. It maps dependencies, applies dynamic relocations, runs initialization machinery, and eventually transfers control through the executable's entry path toward `main`.

This lesson keeps those responsibilities separate:

```
kernel                          user space
──────────────────────────      ──────────────────────────────────────────

accept execve
read ELF/program headers
replace address space
map loadable image + loader
build initial stack
set registers and enter  ────→  dynamic linker entry
                                 map dependencies
                                 resolve/apply relocations
                                 executable _start
                                 libc startup
                                 constructors
                                 main
                                 exit handlers and termination syscall
```

That line is the central discipline for today. “The loader does it” is too vague until we say whether we mean kernel ELF loading, the user-space dynamic linker, or libc startup.

---

1. Sections explain the file; segments explain the process image

Day 5 inspected sections in a relocatable object. They remain useful in an executable:

- `.text` groups instructions;
- `.rodata` groups read-only constants;

- `.data` groups initialized writable objects;
- `.bss` describes zero-initialized storage;
- `.symtab`, `.strtab`, and `.debug_*` support linking and debugging.

But the kernel does not create one mapping per section. It follows the executable's **program header table**. A program header describes a segment: a range of the file and the memory image the runtime needs.

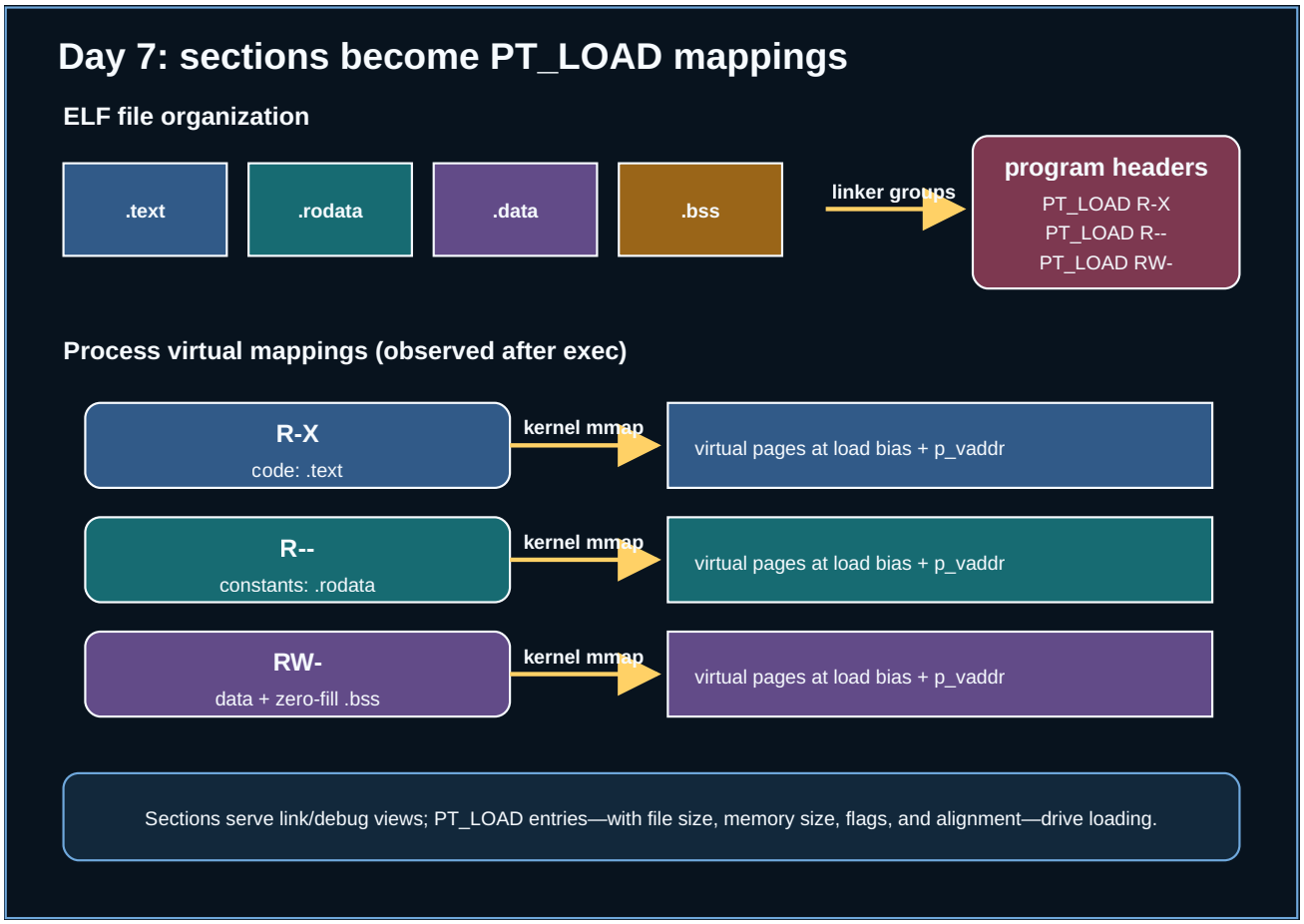
Compare the two views:

```
section header table                  program header table
──────────────────────                ──────────────────────
linker/debugger organization          loading/runtime organization
many named sections                   fewer typed segments
.text, .rodata, .data, ...            PT_LOAD, PT_INTERP, PT_DYNAMIC, ...
may be absent in a stripped file      required information for loading
not the kernel's mapping plan         directly drives the image plan
```

Run:

```
readelf -SW /tmp/startup-probe
readelf -lW /tmp/startup-probe
```

The second command ends with a section-to-segment mapping. Several sections can share one segment because they need compatible file placement and memory permissions.



Sections organize the file for tools; program headers provide the kernel's mapping recipe.

`PT_LOAD`: the mapping recipe

A loadable segment gives the kernel values such as:

```
p_offset   first byte in the file
p_vaddr    virtual address relative to the image's load bias
p_filesz   bytes supplied by the file
p_memsz    bytes required in memory
p_flags    readable, writable, executable
p_align    file/virtual alignment constraint
```

When `p_memsz` exceeds `p_filesz`, the extra memory is zero-filled. This is how `.bss` can occupy memory without equal-sized zero bytes in the file.

Do not translate `PF_R`, `PF_W`, and `PF_X` into an oversimplified promise that every line in `/proc/<pid>/maps` will exactly match one program header. Mappings are page-granular. Segment boundaries can share pages; the dynamic linker can later tighten permissions, notably for GNU RELRO; and kernel, toolchain, and architecture details affect the visible split. The program headers are the requested image, while `/proc/.../maps` is observed runtime state.

Other program headers that matter today

- `PT_INTERP` stores the pathname of the requested ELF interpreter, commonly `/lib64/ld-linux-x86-64.so.2` on x86-64 Fedora.
- `PT_DYNAMIC` identifies dynamic-linking metadata such as `DT_NEEDED`.
- `PT_PHDR` describes the program header table in the memory image.
- `PT_GNU_STACK` communicates requested stack execute permission.
- `PT_GNU_RELRO` marks a range intended to become read-only after relocation.

These are ABI/toolchain contracts layered on ELF. Not every ELF file has each one, and an ELF object need not be a Linux userspace executable.

---

2. `execve` is replacement, not creation

The system call is conceptually:

```
execve(path, argv, envp);
```

It does not create a second process. A shell commonly creates a child first with `fork`/`clone`, and that child calls `execve`, but the exec operation itself replaces the caller's program image.

If `execve` fails, it returns `-1` to the old program with `errno` set. If it succeeds, it does not return to the old instruction stream. The process keeps an identity such as its PID, while large portions of its userspace state are replaced according to exec semantics: mappings, code, stack, signal dispositions, credentials under applicable rules, close-on-exec file descriptors, and other process attributes.

What Linux does

At a deliberately architecture-neutral level, the kernel:

1. validates the executable and execution permissions;

2. selects a binary-format handler, for ELF normally the kernel ELF loader;
3. reads ELF headers and program headers;
4. prepares a new memory-management image;
5. establishes mappings for the executable's `PT_LOAD` ranges;
6. if `PT_INTERP` exists, opens and maps that interpreter too;
7. copies argument and environment strings and constructs the initial stack;
8. adds an **auxiliary vector** with facts needed by libc and the loader;
9. commits the new image and initializes user-mode register state;
10. returns to user mode at the selected entry address.

There is extensive error handling, credential policy, architecture setup, randomization, and security bookkeeping behind that list. The important boundary is simpler:

> The kernel creates the initial executable environment. It does not call the C function `main`, search shared libraries using libc code, or execute C constructors.

Where execution begins

For an ELF file without `PT_INTERP`, the selected entry is the executable's ELF `e_entry`, subject to the image's load bias.

For a normal dynamically linked ELF file, Linux also records the executable's entry for the interpreter and begins user execution at the interpreter's entry. The dynamic linker later transfers control toward the program entry. `readelf -h` shows the executable's `e_entry`; it does not by itself prove that this is the first userspace instruction after a dynamic exec.

This distinction answers a common but imprecise statement:

```
execve succeeds → kernel jumps to main          wrong
execve succeeds → kernel runs the dynamic linker in kernel mode  wrong
execve succeeds → kernel enters prepared user code, often ld.so  precise
```

---

3. The initial stack is a kernel-to-userspace data structure

At process entry, the ordinary C calling convention has not called anything. The ABI defines an initial stack layout from which startup code can recover the process inputs. Conceptually:

```
high addresses
┌──────────────────────────────────────┐
│ argument and environment strings     │
│ platform / random bytes / names      │
├──────────────────────────────────────┤
│ auxiliary vector: type,value pairs   │
│ ...                                  │
│ AT_NULL, 0                           │
├──────────────────────────────────────┤
│ envp[0], envp[1], ... , NULL         │
├──────────────────────────────────────┤
│ argv[0], argv[1], ... , NULL         │
├──────────────────────────────────────┤
│ argc                                 │ ← initial stack pointer region
└──────────────────────────────────────┘
low addresses
```

Exact padding, alignment, register state, and pointer size are ABI-specific. Treat the diagram as relationships, not a byte-perfect universal layout.

`argc`, `argv`, and `envp`

- `argc` counts argument strings.
- `argv` points to an array of pointers terminated by a null pointer.
- `envp` similarly names environment strings, conventionally `NAME=value`.

ISO C standardizes `main` forms with zero or two parameters. A third `char **envp` parameter is a widely supported Unix extension; portable code can use `getenv` or the external environment interfaces instead.

`argv[0]` is supplied by the caller and need not be a trustworthy canonical path. `execve` receives separate `path` and `argv` arguments, so software can choose a surprising `argv[0]`.

The auxiliary vector

The auxv is a sequence of key/value entries provided by the kernel. Useful examples include:

- `AT_PHDR`, `AT_PHENT`, `AT_PHNUM`: the main program's in-memory program header table;
- `AT_ENTRY`: the main executable's entry address;
- `AT_BASE`: the interpreter's load base when applicable;
- `AT_PAGESZ`: system page size;
- `AT_RANDOM`: address of random bytes used by runtime hardening;
- `AT_SECURE`: whether secure-execution rules apply;
- `AT_EXECFN`: the executed pathname string.

The dynamic linker and libc consume these facts without making a syscall for each one. Use `getauxval` in normal C code:

```
unsigned long page_size = getauxval(AT_PAGESZ);
```

`/proc/self/auxv` is binary, not text. `LD_SHOW_AUXV=1` or an appropriate inspection tool can display it for a trusted test program, but `getauxval` is clearer in the lab.

Architecture lens — x86-64, AArch64, and RISC-V

The kernel first checks the ELF machine type, then follows the target's ELF and process-initialization ABI. Interpreter pathnames are conventions rather than one universal name: common examples include `/lib64/ld-linux-x86-64.so.2`, `/lib/ld-linux-aarch64.so.1`, and a RISC-V name that also reflects the selected ABI, but distributions may arrange them differently.

`PT_INTERP` still selects the interpreter and `e_entry` still names an ELF entry, but initial register state, stack alignment, relocation processing, and `_start` code are architecture-specific. Auxv has a broadly shared shape, while entries such as `AT_HWCAP` carry target-specific feature bits; `AT_ENTRY` continues to describe the main executable even when the interpreter runs first. These are target ABI facts beneath the portable hosted-C contract for calling `main`. See the architecture comparison.

---

4. `PT_INTERP` hands control to the user-space dynamic linker

Day 6 stopped with an executable containing `DT_NEEDED` entries and dynamic relocations. `PT_INTERP` tells Linux which program will finish that work.

Inspect it:

```
readelf -lW /tmp/startup-probe | grep -A1 INTERP
readelf -dW /tmp/startup-probe
```

The interpreter—often called `ld.so`, the runtime loader, or the dynamic linker—runs in user mode inside the new process. Broadly, it:

1. discovers the main program and its dynamic metadata;
2. finds required shared objects under loader search and security rules;
3. maps those objects;
4. builds a symbol lookup scope;
5. applies required dynamic relocations;
6. performs TLS and runtime setup;
7. applies protection transitions such as RELRO;
8. arranges initialization and transfers control to the executable entry.

Exact sequencing is implementation- and architecture-dependent. glibc's loader is not the only possible interpreter, and musl or another libc can organize startup differently.

Relocation now has real runtime addresses

ASLR and position-independent objects mean link time cannot know every load base. The dynamic linker computes values using the actual mappings and writes relocation results into designated slots.

Common categories include:

- **relative relocations**, based on an object's load base;
- **symbol-based relocations**, requiring dynamic symbol lookup;

- **PLT/GOT function binding**, either eager or lazy;
- **TLS relocations**, tied to thread-local storage layout.

Relocations do not mean the loader rewrites every instruction. Well-formed PIC uses relative addressing and tables so that most executable code pages remain unchanged and shareable.

Lazy versus eager binding

Some function addresses may be resolved on first call through PLT/GOT state. With eager binding—requested by `LD_BIND_NOW=1` or link options such as `-z now`—they are resolved during startup. Full RELRO commonly combines eager binding with making relevant relocation state read-only.

Do not set loader environment variables globally. Apply them to one trusted command:

```
LD_BIND_NOW=1 /tmp/startup-probe
LD_DEBUG=libs,reloc /tmp/startup-probe 2>/tmp/startup-loader.log
```

Never use `LD_PRELOAD`, `LD_LIBRARY_PATH`, or `LD_DEBUG` casually with untrusted executables or privileged execution. Loader variables alter code selection or reveal detailed paths; secure-execution mode restricts many of them.

---

5. `_start` is an entry point, not a C function

The executable's ELF header names an address. The linker normally obtains the code there from C runtime startup objects—often called **crt objects**— supplied by the toolchain/libc.

```
readelf -h /tmp/startup-probe | grep 'Entry point'
readelf -sW /tmp/startup-probe | grep -E '(_start| main$)'
objdump -d --disassemble=_start /tmp/startup-probe
```

On glibc x86-64, the observed `_start` commonly:

- clears or establishes ABI-required frame state;
- extracts `argc` and the `argv` base from the initial stack;
- aligns the stack;
- passes `main` and startup information to `__libc_start_main`;
- does not expect to return normally.

Do not call `_start` “the first line of every C program.” It is toolchain startup code at an ELF entry, its details differ across libc, architecture, static/dynamic mode, PIE, and toolchain release, and hand-written assembly programs can provide their own entry.

Why `main` needs a runtime before it

Before calling `main`, libc startup may need to:

- initialize libc internal state and threading/TLS support;
- establish stack-protector and other runtime state;
- register finalization machinery;
- coordinate executable and dependency initialization;

- call preinitialization and initialization arrays;
- establish a path from `main`'s return value to process termination.

Some of this overlaps or coordinates with work done by the dynamic linker. The precise internal division is libc-specific. The stable public story is that `main` is reached only after ELF entry code and runtime initialization.

---

6. Constructors run before `main`, but ordering is a contract problem

The lab probe defines:

```
__attribute__((constructor))
static void before_main(void) { /* ... */ }
```

The compiler/linker place an address into an initialization array, normally `.init_array`. Startup machinery walks applicable arrays before `main`.

Inspect:

```
readelf -x .init_array /tmp/startup-probe
readelf -sW /tmp/startup-probe | grep before_main
```

Constructor attributes are compiler extensions. Initialization order among dependencies and the main executable follows ELF/libc rules, but relying on fine-grained order among unrelated constructors is fragile. A constructor:

- runs before the program's ordinary `main` body;
- executes in user mode;
- can call functions only to the extent the runtime is ready;
- can fail, block, allocate, or take locks before application control begins.

Keep constructors small and avoid hidden global-order dependencies.

---

7. `main` returns; libc turns that into orderly process exit

For a hosted C program, returning `status` from the initial invocation of `main` is equivalent in effect to calling `exit(status)`. `exit` performs user-space termination work, including:

- invoking functions registered with `atexit` in reverse registration order;
- flushing and closing C standard I/O streams as specified;
- running applicable finalization machinery;
- finally asking the kernel to terminate the process.

`_exit`/`_Exit` bypass the normal `atexit` and stdio-flush path and proceed toward immediate termination. This matters after `fork`: a child that cannot `exec` often uses `_exit` so it does not flush a copied stdio buffer or run parent-owned cleanup twice.

Keep the layers separate:

```
return from main
  → libc exit path in user space
  → atexit/finalization/stdio cleanup
  → exit_group or equivalent termination system call
  → kernel records status, releases process resources, notifies parent
```

The shell generally displays only the low eight bits of a normal exit status. Signals follow a different wait-status encoding; “exit code 139” is a shell convention commonly representing termination by signal 11, not a program calling `return 139`.

---

8. The lab: inspect without weakening the machine

Run from the repository root:

```
bash 03-c-toolchain-and-startup/challenges/day-007-elf-startup-lab.sh
```

The script builds in a private temporary directory, inspects only its own trusted executable, runs it, captures loader diagnostics to a temporary log, checks the expected status, and removes the artifacts.

Before running, predict:

1. Is the binary `ET_EXEC` or `ET_DYN` on this compiler configuration?
2. How many `PT_LOAD` segments will appear, and which are writable or executable?
3. Which sections share each loadable segment?
4. What pathname will `PT_INTERP` request?
5. Will the constructor message appear before the first `main` message?
6. What values will `argc`, `AT_ENTRY`, `AT_PHDR`, and `AT_PAGESZ` describe?
7. Which mappings belong to the executable, interpreter, libc, heap, stack, vDSO, and vvar?
8. Will the `atexit` handler run before the shell observes status 23?

Read the runtime map as evidence

The probe prints the first entries of `/proc/self/maps`. For a fuller view, temporarily increase its display count or pause it under GDB and inspect:

```
gdb --args /tmp/startup-probe alpha
(gdb) starti
(gdb) info files
(gdb) info proc mappings
```

With a dynamically linked program, `starti` commonly stops in the interpreter, not at the executable's `_start`. Then:

```
(gdb) break _start
(gdb) continue
(gdb) break main
(gdb) continue
```

Record the three instruction addresses and identify their mappings. Debugger behavior can vary depending on GDB and loader integration, so preserve actual output rather than forcing it to match the diagram.

Observe loader decisions safely

The script uses:

```
LD_DEBUG=libs,reloc trusted-program
```

Only stderr receives the loader trace. The lab captures it and prints a bounded excerpt. Read the order:

```
find dependency
  → relocation processing
  → dependency initialization
  → program initialization
  → transfer control
  → finalization
```

`LD_DEBUG` is glibc-loader behavior, not an ELF or Linux-kernel guarantee. If another libc ignores it or formats it differently, that is an environment result to record.

Optional static comparison

If static libc development files are installed:

```
gcc -static -O0 -g \
  03-c-toolchain-and-startup/challenges/day-007-startup-probe.c \
  -o /tmp/startup-probe-static
readelf -lW /tmp/startup-probe-static
```

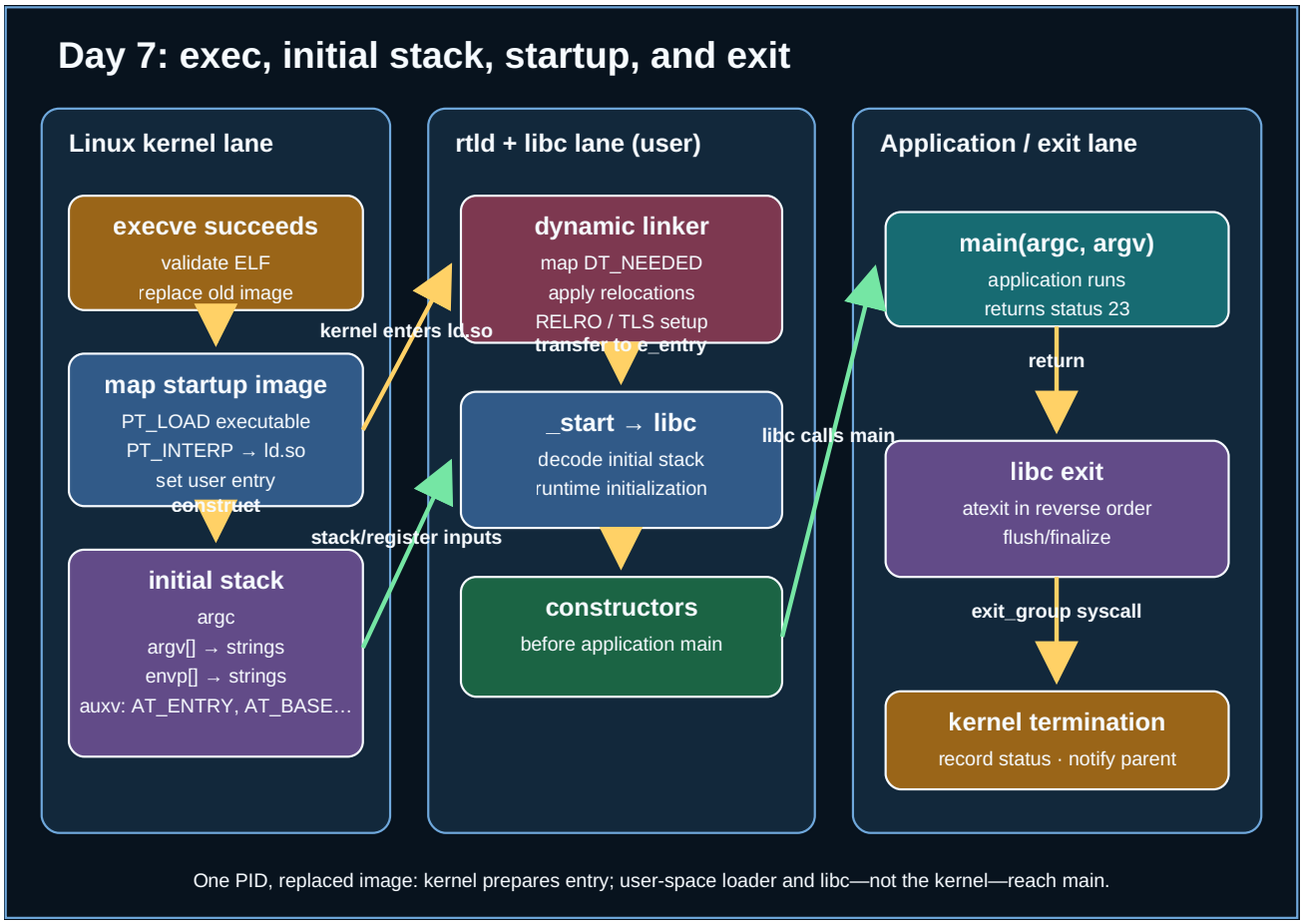
Predict that `PT_INTERP` and shared-library `DT_NEEDED` entries are absent. Do not install packages merely to force this comparison. A static glibc binary still contains substantial libc startup machinery, relocations of its own kinds may remain, and “static” does not mean “the kernel calls `main`.”

---

9. Put the whole route on one page

Trace one successful command:

```
shell
  | prepares argv and envp
  | calls execve
  ▼
Linux kernel
  | validates ELF and reads program headers
  | replaces mappings
  | maps executable PT_LOAD ranges
  | maps PT_INTERP
  | creates initial stack: argc/argv/envp/auxv
  | selects interpreter entry and returns to user mode
  ▼
dynamic linker (user mode)
  | maps DT_NEEDED objects
  | resolves symbols and applies dynamic relocations
  | establishes protections and initialization state
  | transfers toward executable e_entry
  ▼
_start / crt code (user mode)
  | decodes startup state and invokes libc startup
  ▼
libc startup (user mode)
  | runtime initialization and constructor coordination
  ▼
constructors (user mode)
  ▼
main(argc, argv, envp) (user mode)
  | returns 23
  ▼
exit (user mode)
  | atexit handlers, stdio/finalization
  | termination syscall
  ▼
Linux kernel
  | records status and makes it waitable
  ▼
shell observes 23
```



Kernel setup hands one replaced process to user-space startup, which reaches `main` and later returns status through the kernel.

There may be no scheduler switch to a different process at every arrow. “Kernel” and “user” here describe privilege and responsibility, not necessarily different PIDs. After successful exec, this is one process with a replaced userspace image.

---

10. Mastery: Explain · Draw · Observe · Build

The tracker checkpoint is exact:

> **Explain everything that happens after `execve` succeeds and before `main` begins.**

A complete answer must distinguish:

1. kernel validation, image replacement, `PT_LOAD` mappings, interpreter mapping, stack/auxv construction, and user-mode entry;
2. user-space dependency mapping, dynamic symbol lookup, relocations, and protection changes;
3. executable `_start`, crt/libc startup, and constructors;
4. the eventual ABI call to `main`.

Explain

Without notes:

- distinguish sections from segments;
- explain why `PT_LOAD` rather than `.text` directly drives loading;
- explain why successful `execve` does not return;
- identify the first user code for a dynamic executable;
- state what the kernel does **not** do;
- explain why `AT_ENTRY` can name the executable entry even when the interpreter executes first;
- trace `main` returning through `exit`, `atexit`, and the kernel termination boundary.

Draw

Draw:

1. file offsets and sections grouped into loadable segments;
2. the process mappings for executable, interpreter, libc, stack, and vDSO;
3. the initial stack with `argc`, `argv`, `envp`, auxv, and strings;
4. responsibility lanes for kernel, dynamic linker, crt/libc, and application.

Observe

Preserve:

- ELF type, entry, interpreter, and all program headers;
- the section-to-segment mapping;
- `_start` disassembly and `main` symbol;
- constructor and `atexit` ordering;
- selected auxv values;
- `/proc/self/maps` lines labelled by owner and permissions;
- a bounded `LD_DEBUG=libs,reloc` trace;
- optionally, GDB stops in interpreter entry, `_start`, and `main`.

Build

Extend the probe in one of these ways:

1. walk `envp` to the word after its terminating null and decode raw auxv entries, comparing them with `getauxval`; or
2. add two constructors and two `atexit` handlers, predict the order, and test normal `return`, `exit`, and `_Exit` paths; or
3. use `fork` plus one successful and one failed `execve`, proving that only failure returns to the old child image.

Keep every experiment confined to a temporary directory. Do not alter the system interpreter, global loader configuration, ASLR policy, or installed libraries.

Why? notebook

1. Why can an executable retain section headers that the kernel does not need to map it?

2. Why does one `PT_LOAD` commonly contain several sections?
3. Why is `execve` named like a call if success never returns?
4. Why is the dynamic linker ordinary user-mode code despite controlling other mapped code?
5. Why does the kernel supply auxv instead of making libc rediscover every fact?
6. Why is `_start` not called using the normal C function-call convention?
7. Why must constructors precede `main`, and why is complex constructor ordering risky?
8. Why does `_Exit` skip `atexit` handlers?
9. Why can `/proc/self/maps` differ across runs while `readelf -l` does not?
10. Why is `LD_DEBUG` evidence about glibc loader policy rather than kernel ELF loading?

References used selectively

Local:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 7 §§7.9–7.14, PDF pages 730–754, for executable object files, loading, dynamic linking, PIC, and library interposition: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`
- Kerrisk, *The Linux Programming Interface*, Chapter 6 and Chapter 27, for process image layout, argument/environment handling, and program execution: `references/library/books/02-os-and-linux/linux-programming-interface-michael-kerrisk.pdf`

Primary references:

- System V ABI generic ELF specification, program loading and dynamic linking: https://gabi.xinuos.com/
- System V AMD64 ABI, “Process Initialization” and initial stack/auxiliary vector: https://gitlab.com/x86-psABIs/x86-64-ABI
- Linux man-pages `execve(2)`, `getauxval(3)`, `proc_pid_maps(5)`, `exit(3)`, `_exit(2)`, and `ld.so(8)`: https://man7.org/linux/man-pages/man2/execve.2.html https://man7.org/linux/man-pages/man8/ld.so.8.html
- Linux kernel source, `fs/binfmt_elf.c`, for the current ELF binary-format handler. Implementation details are version-specific: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/fs/binfmt_elf.c
- glibc source, `elf/rtld.c`, `csu/libc-start.c`, and architecture startup assembly, for one current libc's user-space implementation: https://sourceware.org/git/?p=glibc.git

End-of-week bridge: Week 3 began with C objects and addresses, crossed lifetime and compiler rules, then followed source through objects and linking. It now ends at the moment those linked bytes become a running process. The next module can study operating-system abstractions without leaving this chain behind.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Elf Startup Lab

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 03-c-toolchain-and-startup/challenges/day-007-elf-startup-lab.sh
```

- View lab source on GitHub

Put it into practice — Startup Probe

Format: C lab

What you will do: Use this startup probe exercise to test the section's model.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 03-c-toolchain-and-startup/challenges/day-007-startup-probe.c -lm -o /tmp/day-007-startup-probe && /tmp/day-007-startup-probe demo
```

- View lab source on GitHub